

Rapport de performance — MicroCRM

Fiche du document	Valeur
Projet	MicroCRM (Orion) — P5, Expert DevOps, « Gérez le cycle de vie de développement logiciel »
Première version	18 août 2026
Mise à jour	2 octobre 2026
Périmètre	chaîne CI/CD, qualité et sécurité du code, déploiement Kubernetes, supervision (logs, traces, alertes), release
Destinataires	le CTO d'Orion, les équipes de développement et d'exploitation, le jury

Ce rapport synthétise ce que le projet a produit, ce qu'il a mesuré, et ce qu'il n'a pas atteint. Il est destiné à un lecteur qui n'ouvrira pas le dépôt : chaque affirmation renvoie au document technique qui la porte, et tous les chiffres cités ont été relevés sur cette installation, jamais estimés.

Le document jumeau, `documentation-infrastructure.md`, décrit l'architecture et les procédures. Celui-ci décrit les résultats.

⚠ **À lire avant tout le reste.** La mise à jour du 2 octobre décrit un travail qui est **dans le dépôt local, pas encore commité ni poussé**. Aucun des jobs ajoutés ou modifiés ce jour-là n'a donc tourné dans un pipeline. Chaque passage concerné le redit, et le §8 en dresse la liste complète.

Synthèse pour décideur

Cette page se lit seule. Les sigles sont expliqués au §10.

Ce qui a été fait

Orion disposait d'une application, MicroCRM, sans chaîne de livraison. Le projet lui en donne une : à chaque modification du code, un **pipeline** (une suite de contrôles automatiques) teste l'application, cherche les failles de sécurité connues, fabrique les **images** (les paquets prêts à déployer), puis les installe sur un environnement de recette (« staging ») et un environnement de production. Une **supervision** rassemble les journaux de l'application dans des tableaux de bord et, depuis le 2 octobre, déclenche des **alertes**.

Ce que ça change pour Orion

- **Une mise en production n'est plus un geste d'expert.** Elle se déclenche d'un clic, et le retour à la version précédente aussi. Les deux ont été joués pour de bon le 23 septembre.
- **Une faille connue arrête la livraison avant la mise en ligne**, et non après. C'est ce qui s'est produit deux fois le 1^{er} octobre.

- **On sait ce que la livraison vaut.** Quatre indicateurs reconnus (DORA) mesurent sa cadence et sa fiabilité, sans rien embellir.
- **Un incident se voit.** Huit règles d'alerte couvrent la disponibilité, la performance et la sécurité ; chacune a été déclenchée volontairement pour prouver qu'elle fonctionne.

Les cinq chiffres qui comptent

Chiffre	Ce qu'il veut dire
5 déploiements réussis sur 9 tentatives	La chaîne livre depuis le 22 septembre, après sept échecs. Six tentatives sur neuf (66,67 %) comptent comme des échecs : elle n'est pas encore fiable.
1,38 h entre une modification et sa mise en ligne	Valeur médiane sur 5 livraisons. Elle contient l'attente du clic humain.
0 faille critique, 5 failles hautes	État de l'image publiée de l'application. Les 5 failles sont corrigées dans le code ; l'image corrigée n'est pas encore publiée.
8 alertes sur 8 déclenchées puis rétablies	Essai du 2 octobre. Les alertes s'affichent dans l'outil de supervision ; elles n'envoient encore ni courriel ni message.
16,1 secondes de coupure	Durée d'indisponibilité si le serveur applicatif tombe. Un déploiement, lui, ne coupe pas le service (99,75 % de requêtes servies pendant un déploiement suivi d'un retour arrière).

Les risques qui restent

1. **La chaîne de livraison est à l'arrêt depuis le 23 septembre.** Les trois derniers pipelines de la branche d'intégration sont en échec. Deux ont été arrêtés, à juste titre, par des failles découvertes dans des composants tiers ; le troisième n'a trouvé aucune machine pour l'exécuter. Les correctifs sont prêts sur le poste, pas encore livrés.
2. **Le travail du 2 octobre n'est pas encore passé dans un pipeline.** Il a été vérifié sur le poste (430 tests de scripts au vert), pas en conditions réelles.
3. **La « production » est une démonstration.** Les deux environnements tournent sur un même ordinateur de développement. Il n'existe pas de production réelle.
4. **Une panne du serveur applicatif coupe le service 16 secondes et efface les données,** parce que la base de données vit en mémoire.
5. **Personne n'est prévenu d'une alerte s'il ne regarde pas l'écran,** et la supervision n'observe que l'environnement de recette.
6. **Douze failles du socle Spring sont acceptées sous condition, jusqu'au 31 décembre 2026.** Leur correctif gratuit n'existe que dans la version majeure suivante du socle.

Les décisions attendues

Décision	Pourquoi maintenant	Détail
Valider la livraison du travail du 2 octobre, puis la publication de la version 1.0.1	C'est ce qui remet la chaîne au vert et met en ligne les correctifs de sécurité	§7.3
Planifier la migration vers Spring Boot 4 avant le 31 décembre 2026	À cette date, les exceptions de sécurité expirent et la livraison se bloque	§4.4
Remplacer la base en mémoire par PostgreSQL	Supprime la coupure de 16 secondes et la perte de données	§7.4
Choisir un canal d'alerte (messagerie, astreinte)	Une alerte que personne ne reçoit ne protège pas	§6.5
Financer un véritable environnement de production	Tout ce qui est mesuré ici l'est sur un poste de développement	§8

1. L'essentiel, y compris ce qui fâche

Le projet livre une chaîne complète. Deux images Docker construites en multi-étapes. Un pipeline GitLab CI de 10 étapes et 39 jobs. Des manifestes Kubernetes en overlays Kustomize, doublés d'un chart Helm. L'infrastructure décrite en Terraform et le poste provisionné par Ansible. Une stack ELK qui collecte réellement les logs de l'application, cinq tableaux de bord et huit règles d'alerte versionnés. Et deux collecteurs écrits pour ce projet : l'un pour les indicateurs DORA, l'autre pour les constats de sécurité.

Cette chaîne a abouti pour la première fois le 22 septembre 2026, après sept échecs. Jusque-là, aucun déploiement n'était parti de la CI, et le dernier pipeline ne s'était pas arrêté sur un test rouge mais sur `ci_quota_exceeded`. Le déblocage tient à un **runner auto-hébergé**, qui ne consomme plus une minute du Free Tier. Il tient aussi à trois correctifs que rien n'avait révélés jusque-là, faute de job allant assez loin : un kubeconfig désignant `127.0.0.1` depuis un conteneur, un RBAC d'agent qui ne couvrait pas les objets applicatifs, et une variable de namespace absente qui faisait déployer dans `default` en silence.

Sur les trente derniers jours, cinq déploiements ont abouti sur neuf tentatives. Staging et production sont posés par la CI avec des images du registry GitLab, et un rollback de production a été joué pour de bon, puis suivi d'un redéploiement. Le taux d'échec des changements reste à **66,67 %** : le chemin automatisé existe désormais, il n'est pas encore fiable, et deux journées d'observations ne font pas une cadence de livraison.

Depuis le 23 septembre, plus rien n'a été déployé. Le pipeline de `develop` est en échec sur ses trois dernières exécutions (29 septembre, deux fois le 1^{er} octobre). Ces trois échecs n'ont pas la même cause, relevée job par job dans l'API GitLab. Le 29 septembre, aucun runner n'était disponible : le runner du projet est un poste de développement, et il n'a pris aucun job. Le 1^{er} octobre à 13 h 20 UTC, `trivy-fs` et `terraform-plan` ont échoué. À 17 h 06 UTC, `package-back` a échoué : le

scan de l'image du back y a trouvé cinq CVE hautes de Jackson. Pour les scans, ce n'est pas une panne de la chaîne, c'est la chaîne qui fait son travail. Le premier échec rappelle autre chose : la livraison dépend d'une machine unique. La conséquence est directe : les images en service datent du 23 septembre, et tout ce qui a été écrit depuis — traces OpenTelemetry comprises — n'est pas déployé.

Le 2 octobre a été une journée de correction, pas de livraison. Le scan des dépendances Java, qui n'analysait aucun jar, en lit maintenant 82. Le scan d'image est fait avant le push et non après. Les règles d'alerte, les tableaux de bord de sécurité et de disponibilité, le job de release et la version 1.0.1 sont prêts. Tout cela est vérifié sur le poste et **rien n'est encore commité** : le §8 dit précisément ce qui reste à prouver.

L'application avait d'abord été déployée à la main, sur un cluster local. Le 10 août 2026, l'overlay `staging` a été appliqué sur minikube. Les deux Deployments ont atteint `rollout status` en `0`, les sondes ont été observées sous kubelet, l'Ingress a routé ses deux hôtes, le rollback automatique a été déclenché sur un échec réel. Un défaut sérieux de la séquence de déploiement a été trouvé puis corrigé. Le compte rendu est dans `K8S.md §14`.

La distinction entre ces deux manières de déployer reste le sujet du §2.4, mais elle a changé de nature. Ce qui était une conviction argumentée — « ce qui manque est le runner, pas la mécanique » — s'est vérifié le jour où le runner est arrivé : le mécanisme n'a pas eu à être réécrit, seuls son accès au cluster et ses droits l'ont été. Ce que la campagne manuelle porte encore seule, c'est le **détail du comportement en exploitation** — rollback sur image volontairement cassée, sondes sous kubelet, perte d'un pod — qu'aucun déploiement de CI ne réexerce.

Tableau de bord du projet

Domaine	Résultat mesuré	Où c'est établi
Pipeline	39 jobs, 10 étapes, toutes images d'outillage figées	ARCHITECTURE.md §4
Tests back	115 tests · 97,40 % lignes · 100 % branches	back/ , job test-back
Tests front	112 tests · 100 % lignes · 90,24 % branches	front/ , job test-front
Tests des scripts	430 assertions, sans cluster ni registry	SCRIPTS.md
Validation des manifestes	151 assertions, dont l'équivalence Kustomize ↔ Helm	SCRIPTS.md , HELM.md §6
Playbook Ansible	<code>ok=23 changed=0</code> sur deux exécutions consécutives	ANSIBLE.md §7
Terraform	3 environnements, <code>validate</code> et <code>plan</code> en <code>0</code>	TERRAFORM.md

Domaine	Résultat mesuré	Où c'est établi
Latence du front	p50 0,14 ms · p95 1,60 ms · p99 3,11 ms (105 req.)	MONITORING.md §5
Latence de l'API	p95 de 3,41 à 5,65 ms selon la route — conteneur local (§5.3)	MONITORING.md §10.4
Logs collectés	92 175 documents au 2 octobre, 100 % du namespace de staging	k8s/elk/alerting/ README.md
Alertes	8 règles , 3 familles, 8 déclenchées puis rétablies	k8s/elk/alerting/ README.md
Constats de sécurité	0 CRITICAL, 5 HIGH sur l'image publiée du back	k8s/elk/dashboards/ README.md
Disponibilité en déploiement	99,75 % sur 1 615 requêtes, coupure ≤ 0,20 s	K8S.md §14.10
Déploiements réussis en CI	5 sur 9 tentatives (30 j)	MONITORING.md §9

2. Les indicateurs DORA

2.1 Les quatre indicateurs, et ce qu'ils signifient

Les indicateurs DORA (du nom du programme de recherche *DevOps Research and Assessment*) sont quatre mesures qui décrivent la livraison logicielle par ses résultats. Deux mesurent la **vitesse**, deux la **stabilité** ; les lire ensemble empêche d'améliorer l'une en dégradant l'autre.

Indicateur	Ce qu'il mesure	Comment il est calculé ici	Ce qu'il dit à Orion
Fréquence de déploiement	Combien de fois par jour une version arrive en service	Jobs de déploiement réussis, divisés par les 30 jours de la fenêtre	La cadence réelle de livraison
Délai de mise en production	Le temps entre une modification et sa mise en service	Médiane, du commit de tête à la fin du job de déploiement réussi	La réactivité : combien de temps attend un correctif
Temps de rétablissement (MTTR)	Le temps pour retrouver un service sain après un échec	Médiane, d'un déploiement en échec au déploiement réussi suivant	La capacité à se relever

Indicateur	Ce qu'il mesure	Comment il est calculé ici	Ce qu'il dit à Orion
Taux d'échec des changements	La part des déploiements qui échouent ou qu'il faut annuler	Échecs directs et réussites suivies d'un rollback, divisés par les tentatives	La fiabilité de ce qu'on livre

Pourquoi ceux-là. Ils se calculent depuis l'historique des pipelines, que le projet produit de toute façon : aucune saisie manuelle, donc aucun chiffre arrangé. Et ils mesurent ce qui intéresse Orion — des versions qui arrivent en service sans casse — plutôt que l'activité de l'équipe.

2.2 Comment ils sont produits

Les quatre indicateurs sont calculés par `scripts/ci/collect_dora.py`, en Python standard et sans aucune dépendance. Les métriques DORA natives de GitLab sont réservées aux offres payantes ; sur le Free Tier, il faut les calculer soi-même depuis l'API. Le projet est mesurable **sans jeton**, parce que le dépôt GitHub se miroite vers un projet GitLab public où le pipeline tourne réellement.

Le collecteur est testé sur des **fixtures**, c'est-à-dire des réponses d'API enregistrées, et non contre le réseau : un test qui dépend d'un service tiers échoue les jours où ce service est lent, et on finit par ne plus le croire. Deux jeux coexistent, et la distinction est délibérée. Le premier est réel, enregistré verbatim avant le déblocage du 22 septembre : les 44 pipelines d'alors et les jobs des 7 pipelines ayant déclenché un déploiement. Le second, explicitement nommé `dora-scenario-fabrique`, contient ce que le jeu réel n'offre pas : des déploiements réussis. Sans lui, les formules du délai et du temps de rétablissement ne seraient empruntées par aucun test, et on ne vérifierait qu'une chose — la capacité du collecteur à dire « je n'ai rien ». Les fixtures réelles n'ont pas été réenregistrées depuis ; elles décrivent toujours correctement la fenêtre qui est la leur.

2.3 Les valeurs : mesurées le 23 septembre 2026, rejouées le 2 octobre

Indicateur	Valeur	Observations
Fréquence de déploiement	0,1667 par jour	5
Délai de mise en production (médiane)	1,38 h (min 0,64 / max 4,87)	5
Temps de rétablissement (MTTR, médiane)	2,14 h (min 0,09 / max 4,18)	2
Taux d'échec des changements	66,67 %	9

Mesurées le 23 septembre sur 50 pipelines, ces valeurs ont été recalculées le 2 octobre contre l'API réelle, sur **57 pipelines** : elles sont identiques, parce qu'aucun déploiement n'a eu lieu entre-temps. Neuf jobs de déploiement ont tourné sur la fenêtre, cinq ont réussi, et deux rollbacks ont été joués.



Ce qu'il faut y lire : les quatre tuiles du haut sont les quatre indicateurs ; la chronologie montre que toute l'activité tient sur deux journées, et que rien n'a été déployé depuis. La table du bas liste chaque déploiement et chaque rollback, un par ligne.

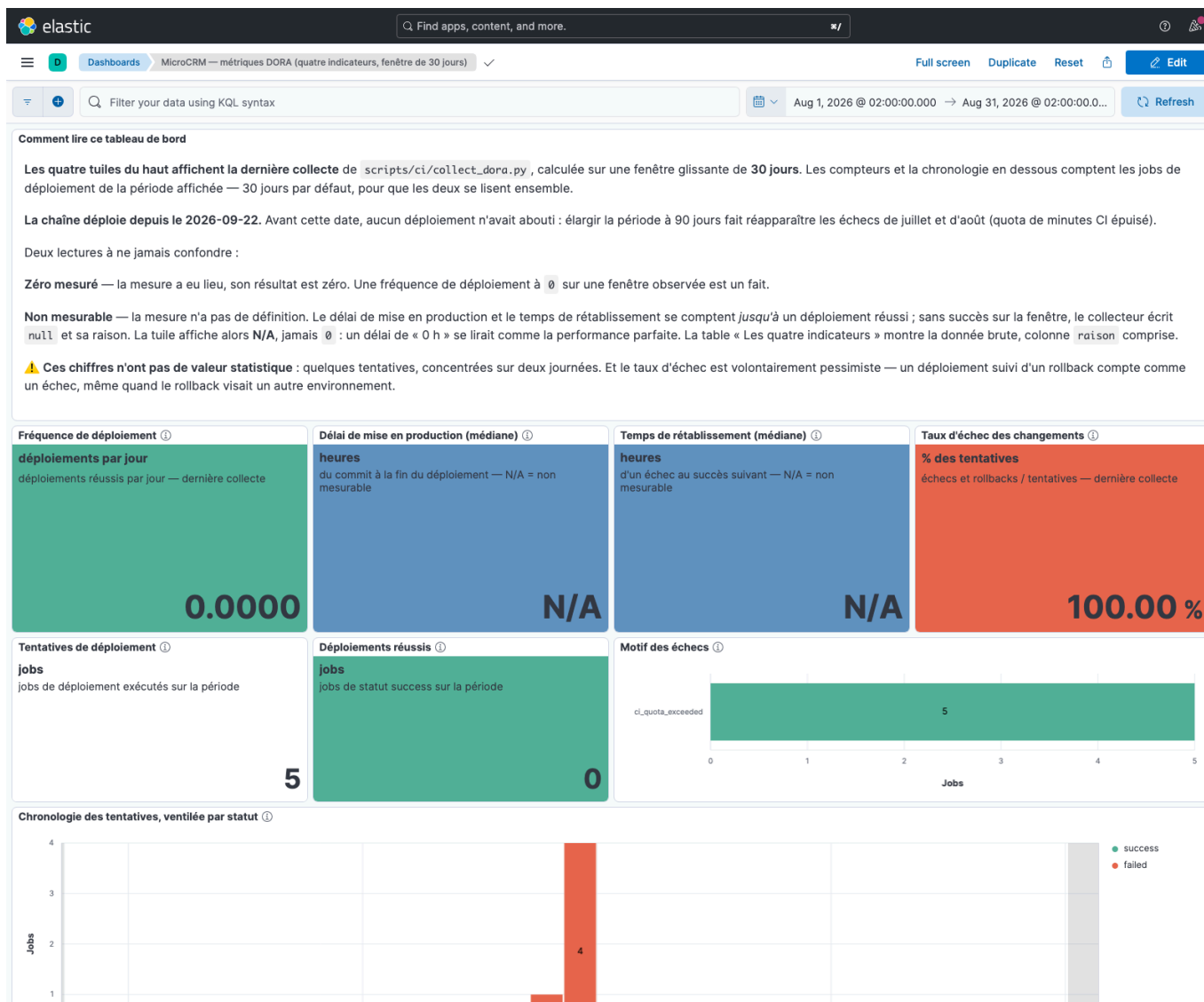
Une lecture honnête de ces quatre valeurs tient en quatre remarques.

Le recul est de deux jours, pas de trente. Les cinq déploiements réussis sont concentrés sur les 22 et 23 septembre ; la fréquence de 0,1667 par jour est donc une moyenne sur une fenêtre dont l'essentiel est vide. Un taux d'échec calculé sur 9 tentatives et un MTTR sur 2 observations décrivent des faits, pas des tendances.

La fenêtre glisse, et ces valeurs vont se dégrader toutes seules. Sans nouveau déploiement, les cinq réussites sortiront de la fenêtre de 30 jours à partir du 22 octobre : la fréquence retombera à 0 et le délai redeviendra non mesurable. C'est l'effet attendu d'un indicateur glissant, et une raison de plus de remettre la chaîne au vert.

Le délai de mise en production mesure surtout un geste humain. Les deux jobs de déploiement restent `when: manual` : la médiane de 1,38 h entre le commit de tête et la fin du déploiement contient l'attente du clic, pas seulement la durée du pipeline.

Zéro mesuré et absence de donnée ne sont pas la même chose. C'est la règle qui gouverne tout le collecteur, et la seule qui le rende utile plutôt que décoratif. Pendant les deux mois sans déploiement, `deployment_frequency` valait `0.0` — une mesure — tandis que `lead_time_for_changes` valait `null`, accompagné de sa raison : il n'existait aucune arrivée en production vers laquelle mesurer un délai. Le rendre en `0` aurait affiché un délai nul, c'est-à-dire **la performance parfaite**, là où il n'y avait jamais eu de mise en production. Un test de `run_tests.sh` échoue si un indicateur sans donnée se met à ressortir en `0`, et le tableau de bord respecte la même règle :



Ce qu'il faut y lire : sur une période sans aucun succès, les deux indicateurs qui n'ont pas de définition affichent « N/A », jamais « 0 ».

⚠ Trois limites à connaître sur cette collecte.

- **Elle n'a encore tourné dans aucun pipeline.** Un job `dora-metrics` a été écrit le 2 octobre (`.gitlab/ci/deploy.yml`) : il exécute le collecteur sur `develop`, `main` et les tags, et publie `reports/dora.json` en artefact. Il n'est pas commité. Le script, lui, a tourné sur le poste (code 0 en 70 s).
- **Le job n'alimente pas le tableau de bord.** Il n'envoie rien à Elasticsearch, qu'un runner ne peut pas joindre ici. L'index `microcrm-dora` a été réinjecté à la main le 2 octobre ; il ne l'avait plus été depuis le 16 août.
- **Le comptage des rollbacks est pessimiste, et assumé tel quel.** Un déploiement réussi puis annulé compte comme un échec, conformément à la définition DORA, **mais le rattachement d'un rollback à un déploiement est purement chronologique**. Le rollback du 23 septembre à 13:01:30 a échoué sur un timeout et n'a donc rien annulé ; il est quand même compté comme une annulation. Le détail, et ce qu'il faudrait pour le corriger, sont dans `MONITORING.md` §9.3.

La liste des pipelines, le 23 septembre 2026, après les déploiements :

Ted Pasquie / G-rez-le-cycle-de-vie-de-d-veloppement-logiciel / Pipelines

All 94

Finished

Branches

Tags

View analytics

Clear runner caches

New pipeline

Filter pipelines

Show Pipeline ID

Status	Pipeline	Created by	Stages	Actions
Blocked 00:20:29	#2874278876 Merge pull request #25 from TedPasqui... main → 9f4168b3 latest branch		✓✓✓✓✓✓✓ ✓⊛⊛→✓	▶ ⊛ ⬇
Blocked 00:16:27	#2874277560 Merge pull request #24 from TedPasqui... develop → bf272532 latest branch		✓✓✓✓✓✓✓ ✓⊛→✓	▶ ⊛ ⬇
Blocked 00:22:22	#2873813689 Merge pull request #23 from TedPasqui... main → 819da793 branch		✓✓✓✓✓✓✓ ✓⊛⊛→✓	▶ ⊛ ⬇
Passed 00:24:34 14 hours ago	#2872416415 Merge pull request #22 from TedPasqui... develop → 556e01df branch		✓✓✓✓✓✓✓ ✓✓→✓	▶ ⬇
Failed 00:24:01 15 hours ago	#2872294040 Merge pull request #21 from TedPasqui... develop → e9bcb9da branch		✓✓✓✓✓✓✓ ✓⊛→✓	▶ ↺ ⬇
Failed 00:07:25 17 hours ago	#2871734734 Merge pull request #20 from TedPasqui... develop → 778de280 branch		✓✓✓✓⊛✓»» »»	▶ ↺ ⬇
Canceled 00:00:00 18 hours ago	#2871733658 Merge pull request #19 from TedPasqui... develop → b89612d0 branch		⊛⊛⊛⊛⊛⊛⊛ ⊛⊛	▶ ↺ ⬇
Canceled 00:00:32 18 hours ago	#2871730220 docs(release): consigne la restauration j... feature/restauration-jouee b5913604 latest branch		✓⊛⊛⊛	↺ ⬇

⚠ « Blocked » n'est pas un échec, et c'est la première chose à lire sur cette capture. C'est le statut d'un pipeline dont tous les jobs automatiques ont réussi et qui attend une action manuelle — ici les `terraform-apply` et les jobs de déploiement, qui sont des portes volontaires (§2.4). Un pipeline en échec porte « Failed » et une pastille rouge — deux d'entre eux sont visibles plus bas dans la liste. Les trois pipelines du haut — #2874278876 (main, 9f4168b3), #2874277560 (develop, bf272532) et #2873813689 (main, 819da793) — sont ceux qui portent toute l'activité du 23 septembre : quatre déploiements réussis, un échec et les deux rollbacks.

2.4 « Ça marche sur un cluster local » n'est pas « ça arrive en production »

C'est la différence que ces quatre chiffres mesurent, et il vaut mieux l'énoncer que la laisser deviner.

Ce qui a été **prouvé sur un cluster** : les manifestes s'appliquent, les pods démarrent, les sondes se comportent comme prévu sous un vrai kubelet, l'Ingress route par hôte, le CORS discrimine, un déploiement vers une image inexistante déclenche bien le rollback automatique, et un rollback

ramène la version précédente réellement déployée. Tout cela a été observé, commande par commande.

Ce qu'un déploiement local ne pouvait pas exercer a depuis été exercé : un runner, un registry privé authentifié, des variables protégées, l'accès au cluster par le tunnel de l'agent, et un enchaînement que personne ne pilote entre le clic et le rollout. Le tirage d'image par `imagePullSecrets` fonctionne : les pods de staging et de production tirent leurs images du registry GitLab.

Ce qui **n'a toujours pas lieu** : qu'un commit poussé sur `develop` ou `main` se retrouve en marche sans intervention humaine. Les deux jobs de déploiement sont `when: manual`, et c'est un choix pour la production (`A3.1` du plan d'optimisation propose de le lever sur staging seulement).

Ce que la campagne locale porte encore seule : le comportement sous incident. Les quatre déploiements de CI qui ont échoué l'ont fait sur l'accès au cluster — kubeconfig, droits, namespace — et non sur l'application ; aucun n'a donc exercé une image cassée, la perte d'un pod ou une sonde en échec. La différence entre les deux registres s'est donc réduite, elle n'a pas disparu — et **les indicateurs DORA restent l'instrument qui refuse de confondre « le mécanisme fonctionne » et « les changements arrivent en production sans casse »**. C'est la raison pour laquelle ils sont présentés ici tels quels, sans correctif ni explication de rattrapage : à 66,67 % d'échec, ils ne flattent personne.

3. Tests et couverture

3.1 Les quatre suites

Suite	Volume	Couverture	Job CI
Back (JUnit, JaCoCo)	115 tests	97,40 % lignes · 100 % branches	test-back
Front (Karma, LCOV)	112 tests	100 % lignes · 90,24 % branches	test-front
Scripts d'automatisation	430 assertions	—	test-scripts
Manifestes et chart	151 assertions	—	lint-helm

Les volumes des deux dernières lignes ont été remesurés le 2 octobre sur l'arbre de travail, donc avec le travail non commité : `run_tests.sh` rend « 430 test(s) OK, 0 en échec », `validate_k8s.sh` « 151 test(s) OK », et 174 avec `--autotest`. Les 112 tests du front et leur couverture viennent d'une exécution locale de `ng test` le même jour. Les 115 tests du back ont été rejoués le même jour ; leur couverture n'a pas été remesurée.

L'écart entre lignes et branches était le chiffre intéressant du back ; il est refermé : **100 % des embranchements** sont désormais empruntés, contre 62,5 % auparavant. Une couverture de lignes élevée ne dit rien des chemins d'erreur, et c'est là que se logeait le risque. Le seuil dur du pipeline (`COVERAGE_MIN`) est fixé à **90 %** de lignes, contrôlé par `scripts/ci/check_coverage.py` dans

le job `coverage-gate`, **bloquant** — un contrôle rapide, indépendant de la disponibilité de Sonar. Les tests de mutation (`MUTATION_MIN`, 80 %) posent la question que la couverture ne sait pas poser : ces tests vérifient-ils quelque chose ?

3.2 Ce que les tests d'infrastructure vérifient réellement

Deux suites ne testent pas du code applicatif, et ce sont celles qui ont le plus de valeur ici.

Les 430 assertions de `run_tests.sh` exercent les scripts de déploiement sans cluster ni registry : les vraies commandes `kubectl`, `docker`, `trivy` et `k6` sont remplacées par de faux programmes qui notent ce qu'on leur demande et renvoient le code de sortie voulu. On vérifie ainsi qu'un `rollout undo` est réellement déclenché quand un déploiement échoue, qu'il ne l'est **pas** quand tout va bien, que le scan d'une image précède son push, qu'aucune image n'est poussée si Trivy y trouve une CVE haute ou critique, et qu'aucun secret n'apparaît dans les logs. Depuis le 2 octobre, elles couvrent aussi le collecteur de sécurité (56 assertions) et l'installation des alertes (33).

Les 151 assertions de `validate_k8s.sh` portent sur ce que Kustomize et Helm produisent réellement. La plus importante est la dernière : **le rendu du chart Helm est comparé objet par objet à celui de l'overlay Kustomize correspondant**, et l'écart n'est toléré que sur le seul label `app.kubernetes.io/managed-by`. Deux descriptions de la même application, c'est deux occasions de diverger ; c'est cette assertion qui rend la divergence visible en revue plutôt qu'au déploiement.

`validate_k8s.sh` est **auto-testé** : l'option `--autotest` rejoue ses assertions sur des rendus volontairement abîmés — Deployment renommé, ConfigMap fantôme, sonde sur un port inconnu, `runAsNonRoot` retiré, image en `latest` — et vérifie qu'elles échouent bien. De même, les nouvelles assertions de pipeline de `run_tests.sh` ont été vérifiées en échec sur une copie abîmée. Une assertion qui ne se déclenche jamais ne prouve rien.

3.3 Ce que les tests ont trouvé

Trois trouvailles méritent d'être citées, parce qu'elles justifient le coût de ces suites.

- **`POST /persons` renvoyait un corps vide.** Signalé dès la première exécution du test de fumée k6. Ce n'était pas un bug de l'API : Spring Data REST ne renvoie la ressource créée que si le client envoie un en-tête `Accept`. Le front Angular l'envoie, k6 non. Le scénario a été corrigé pour se comporter comme le vrai client — un test qui n'aurait vérifié que le code HTTP aurait laissé passer l'écart.
- **`rollback-production` ne pouvait pas fonctionner.** Découvert lors de la campagne sur cluster réel : chaque déploiement insérait **deux** révisions — un `apply -k` posant l'image `placeholder`, puis `deploy.sh` posant la vraie — si bien que la « révision précédente » d'un déploiement sain était toujours le placeholder. Le seul geste de secours prévu par le pipeline visait donc une image inexistante. Corrigé par un overlay Kustomize éphémère qui pose l'image réelle **avant** l'`apply`, et vérifié sur cluster : deux déploiements, deux révisions, zéro placeholder, rollback sans argument en `0`.
- **La suppression d'un pod back coupe l'API 16,1 secondes.** Non détecté par la première campagne, qui décrivait l'état avant et après sans mesurer l'intervalle. Voir §5.4.

4. Qualité et sécurité du code

4.1 Les contrôles, et lesquels arrêtent le pipeline

Six outils sont branchés sur le cycle de vie, chacun voyant ce que les autres ne voient pas : un test unitaire ne détecte pas une CVE, un scan de CVE ne détecte pas une mauvaise pratique de code, et aucun des deux ne dit si l'application tient la charge.

Outil	Question à laquelle il répond	Étape	Bloquant ?
Checkstyle, Spotless, ESLint, Prettier, ShellCheck	La forme est-elle tenue ?	lint	oui
SonarQube	Bonnes pratiques, dette, couverture agrégée	quality	oui
SpotBugs + Find-Sec-Bugs	Bugs latents dans le bytecode (~140 motifs sécurité)	quality	oui
OWASP Dependency-Check	CVE des dépendances Java (base NVD)	security	oui, score CVSS ≥ 7
Trivy	CVE du dépôt et des images, secrets, misconfigurations	security et package	oui, HIGH et CRITICAL
k6	L'API répond-elle, et assez vite ?	perf	oui pour le smoke, non pour la charge

Tous ces contrôles sont bloquants, à une exception près. La version précédente de ce tableau marquait SonarQube et SpotBugs « non » : c'était périmé. Relu dans `.gitlab/ci/quality.yml`, aucun des jobs `sonar-back`, `sonar-front`, `spotbugs-back` et `mutation-back` ne porte `allow_failure`, et un job GitLab sans cette clé est bloquant par défaut. L'état de départ — six jobs en `allow_failure: true` et des scans Trivy en `--exit-code 0` — a été levé en deux temps : les contrôles de qualité et `dependency-check-back` le 14 septembre 2026, les trois scans Trivy le 19 septembre. Le choix de démarrage était assumé ; il n'avait pas vocation à être présenté comme final.

Une précision sur SonarQube : les deux analyses tournent à chaque pipeline, mais le job `quality-gate`, qui interroge le verdict de la *Quality Gate* de Sonar, ne tourne que sur `main`.

Un seul contrôle reste non bloquant : `k6-load`. Ses mesures varient d'une exécution à l'autre sur un runner mutualisé, et un seuil dur y produirait des échecs sans rapport avec le code. Sur un runner dédié, il suffit de basculer son `allow_failure`. `k6-smoke`, lui, a toujours été bloquant, parce qu'il ne mesure pas une tendance mais un fait binaire : l'image qu'on s'apprête à déployer répond, ou elle ne répond pas. Deux autres jobs portent `allow_failure: true`, `notify-echec` et `dora-metrics` : ce ne sont pas des contrôles, et une notification ou une mesure en panne ne doit pas faire échouer une livraison.

Deux constats connus et non corrigés : Find-Sec-Bugs remonte `PERMISSIVE_CORS` dans `SpringDataRestCustomization.java`, et **aucun job ne lance `npm audit`** — vérifié le 2 octobre, la commande n'apparaît dans aucun fichier du pipeline. Les dépendances du front ne sont couvertes que par `trivy-fs`, qui lit le `package-lock.json`. C'est une piste identifiée, pas un contrôle en place.

4.2 Ce que la journée du 2 octobre a corrigé

Trois défauts ont été trouvés en cherchant pourquoi `develop` était rouge. Les trois affaiblissaient un contrôle de sécurité sans que rien ne le signale. Les correctifs sont sur le poste, vérifiés localement, **non commités**.

Dependency-Check n'analysait aucun jar. Le rapport du job `dependency-check-back` contenait une liste de dépendances **vide**, et Gradle y passait cinq secondes. Le contrôle était bloquant, vert, et ne regardait rien. La cause est une rencontre de deux conventions : le plugin écarte par défaut les configurations « de test », qu'il reconnaît à leur nom, et Spring Boot fait hériter `runtimeClasspath` d'une configuration nommée `testAndDevelopmentOnly`. La seule configuration demandée était donc écartée. Une ligne corrige cela (`skipTestGroups = false` dans `back/build.gradle`). Le premier scan réel a lu **82 dépendances** et relevé **13 CVE de score ≥ 7** : douze sur Spring Framework 6.2.19, une sur Log4j. C'est aussi l'explication d'un écart resté longtemps sans réponse : Trivy trouvait des CVE que Dependency-Check ne voyait pas, parce que Dependency-Check ne voyait rien.

L'image était poussée avant d'être scannée. Le scan Trivy était une ligne placée après `build_and_push.sh`, donc après le push. Quand le scan refusait l'image, le job passait au rouge — mais l'image était déjà au registry sous son SHA. C'est le cas de `back:5bf1d6a2`, publiée avec ses cinq CVE hautes. Or la promotion d'une release ne vérifie que l'existence de ce tag : un tag de version posé sur ce commit aurait promu une image refusée. Le scan est désormais fait **dans `build_and_push.sh`**, entre la construction et le push (option `--scan`, activée dans `package-back` et `package-front`), et un test vérifie cet ordre.

Les scans ne laissent aucune trace exploitable. Un job rouge disait « une CVE a été trouvée », et il fallait relire son journal pour savoir laquelle. Le nouveau script `scripts/ci/trivy_scan.sh` fait deux passages : un relevé au format JSON, puis la porte bloquante sous forme de tableau lisible. Les jobs `trivy-fs`, `package-back` et `package-front` publient ces rapports en artefacts (`reports/`, conservés une semaine, y compris quand le job échoue). Son code de sortie distingue enfin un constat (2) d'un scan impossible (1).

4.3 L'état des vulnérabilités au 2 octobre 2026

Le collecteur `scripts/ci/collect_security.py` lit les rapports JSON de Trivy et de Dependency-Check, ainsi que les exceptions de `.trivyignore.yaml`, et les envoie dans l'index Elasticsearch `microcrm-security`. Le tableau de bord « sécurité » (§6.4) en est la lecture. Dernier scan de chaque source, toutes sévérités :

Source	Ouverts	CRITICAL	HIGH	MEDIUM	LOW	Exceptés
Dépôt (code, manifestes, configuration)	58	0	0	21	37	11
Image du back (<code>back:5bf1d6a2</code>)	12	0	5	7	0	0
Image du front (<code>front:5bf1d6a2</code>)	10	0	0	5	4	0
Total	80	0	5	33	41	11

Le total de 80 compte aussi un constat de sévérité inconnue sur l'image du front. Par nature : 58 défauts de configuration, 22 vulnérabilités, aucun secret.

Les cinq constats HIGH sont identifiés, priorisés et corrigés dans le code. Ce sont cinq CVE de `jackson-core` et `jackson-databind` 2.21.4, toutes dans le jar de l'application. Le correctif force Jackson à 2.21.7. Vérifié le 2 octobre : les images back et front, construites sur le poste en version 1.0.1 puis scannées, sortent à **0 constat HIGH ou CRITICAL**. Mais ces images de vérification ont été supprimées, et **aucune image corrigée n'existe au registry** : le tableau de bord montre l'image publiée, pas le code.

Le chemin suivi par ces cinq CVE est celui qu'on attend d'une chaîne DevSecOps : détectées par le scan d'image, elles ont arrêté le pipeline avant tout déploiement, et elles n'ont jamais atteint staging ni production — dont les images, antérieures, ont été scannées et acceptées le 23 septembre.

4.4 Les exceptions : assumées, justifiées, datées

Une exception est une vulnérabilité signalée que l'on décide de ne pas corriger tout de suite. Elle n'est acceptable que justifiée et bornée dans le temps. Toutes celles du projet expirent le **31 décembre 2026** : passé cette date, le contrôle se referme et impose la relecture.

Fichier	Exceptions	Portée
<code>.trivyignore.yaml</code>	7 entrées, couvrant 11 constats	défauts de configuration de manifestes, justifiés un à un
<code>back/config/dependency-check/suppressions.xml</code>	3 entrées, couvrant 12 CVE Spring Framework	modules, fonctions ou conditions que l'application ne réunit pas

Les douze CVE de Spring Framework 6.2.19 méritent d'être détaillées, parce que **aucune montée de version gratuite ne les lève**. La version corrigée de la branche 6.2, la 6.2.20, est réservée au support payant de l'éditeur ; le seul correctif en source ouverte est Spring Framework 7, c'est-à-dire Spring Boot 4, une migration majeure. Attendre ne sert donc à rien.

- **Six CVE** visent WebFlux et RSocket, des modules absents du jar livré.
- **Trois CVE** visent des fonctions de Spring MVC que l'application n'utilise pas : elle ne déclare aucun contrôleur, seulement deux dépôts Spring Data REST.

- **Trois CVE** (CVE-2026-47886, CVE-2026-59282, CVE-2026-59283) touchent au langage d'expression SpEL et à la liaison de données. Elles avaient d'abord été laissées bloquantes, faute d'analyse. L'analyse a été faite le 2 octobre, condition par condition, contre les avis de l'éditeur : pour chacune, au moins une des conditions exigées manque. Elle a été doublée d'un contrôle par exécution sur le back de staging : trois requêtes `PATCH` forgées ont chacune répondu `400` en moins de 0,1 s, sans effet sur la ressource.

⚠ **Ce que ces exceptions ne valent pas.** Elles ne prouvent pas que Spring 6.2.19 est sain : elles disent que les conditions d'exploitation ne sont pas réunies aujourd'hui. Elles tombent le jour où le modèle de données reçoit un champ `BigDecimal`, ou où du code évalue une expression venue d'une requête. La sortie propre est la migration vers Spring Boot 4. Le commentaire de `suppressions.xml` porte l'analyse complète.

⚠ **La porte passe sur le poste ; elle reste à constater en CI.** Le rapport local du 2 octobre (`back/build/reports/dependency-check-report.json`, 21 h 31) porte 82 dépendances, 12 CVE écartées par exception et aucune CVE ouverte de score ≥ 7 ; relancée, la tâche Gradle sort en `0`. Aucun pipeline n'a encore exécuté `dependency-check-back` dans cet état : c'est lui qui dira si `develop` repasse au vert.

5. Performance mesurée

5.1 Le budget de performance et les scénarios k6

Trois scénarios, écrits en JavaScript et versionnés à côté du reste, s'exécutent **après** l'étape `package` : GitLab démarre l'image Docker qui vient d'être construite comme un service, et k6 l'interroge. On mesure donc l'artefact qui partirait réellement en production, pas une compilation locale.

Scénario	Charge	Rôle	Bloquant
<code>smoke.js</code>	1 utilisateur, 5 parcours complets	« est-ce que ça marche ? »	oui
<code>load.js</code>	10 utilisateurs en lecture + 2 écritures/s	garde-fou des temps réponse	non
<code>stress.js</code>	paliers jusqu'à 50 utilisateurs, sans pause	trouver le point de rupture	déclenché à la main

Seuil	Valeur par défaut
p95 des lectures	500 ms
p95 des écritures	800 ms
Taux de requêtes en erreur	< 1 %
p95 du smoke (application froide)	1 500 ms

Deux choix structurent ces seuils. **Des percentiles et pas des moyennes** : une moyenne correcte peut très bien cacher 5 % d'utilisateurs qui attendent trois secondes. Et **un seuil par endpoint** plutôt qu'un seuil global, ce qui permet de dire « c'est la recherche par email qui décroche » au lieu de « c'est lent ». Chaque réponse est en outre vérifiée fonctionnellement, statut et contenu : un serveur qui renvoie 500 en 3 ms est très rapide et totalement cassé.

Les seuils vivent dans le dépôt et **ne sont pas exposés en variables GitLab** : un garde-fou dont on desserre le seuil depuis une interface pour faire passer un pipeline rouge n'est plus un garde-fou. Les variables CI règlent la charge, pas le budget.

⚠ **Ces mesures dépendent de la machine.** Elles servent à détecter une régression entre deux exécutions comparables, pas à annoncer une capacité absolue. La base étant une HSQLDB en mémoire, plus rapide qu'une base réseau, les chiffres sont optimistes en valeur absolue. Le front n'est pas testé en charge : seul le back l'est, parce que c'est lui qui porte le risque de saturation.

5.2 La latence du front, mesurée en service

Il n'existait au départ **aucune donnée de latence dans toute la chaîne**. Les logs applicatifs du back portent ce que l'application raconte, pas le temps qu'elle met ; et le Caddyfile n'avait aucune directive `log`. Un écran « latence » construit là-dessus aurait été décoratif. Le journal d'accès de Caddy a donc été activé — c'est la seule source qui voit passer le trafic du front, et elle porte les trois mesures d'un coup.

Percentile	Latence mesurée (105 requêtes, vérification initiale)
p50	0,14 ms
p95	1,60 ms
p99	3,11 ms

Sur la durée, la mesure tient : sur 624 tranches de cinq minutes relevées en sept jours, le p95 médian est de **1,54 ms**, et 90 % des tranches sont sous 2,75 ms (`k8s/elk/alerting/README.md`). Douze tranches dépassent 100 ms ; les quatre qui comptent au moins 20 requêtes correspondent toutes à des moments où le poste était saturé.

⚠ **C'est la latence vue par le serveur web, pas par l'API.** Caddy sert le bundle Angular et `/config.json` ; les appels à l'API partent du navigateur vers un hôte distinct et ne passent pas par lui. Et l'essentiel de ce trafic est fait des sondes de Kubernetes, pas d'utilisateurs.

⚠ **Le front ne produira quasiment jamais de 4xx**, parce que toute route inconnue renvoie `200` avec `index.html`, à charge pour le routeur Angular de décider. Vérifié : 12 requêtes vers des chemins inexistants, 12 réponses `200`. Les erreurs réelles se lisent côté back.

5.3 La latence de l'API : mesurable depuis le 1^{er} octobre, pas encore en service

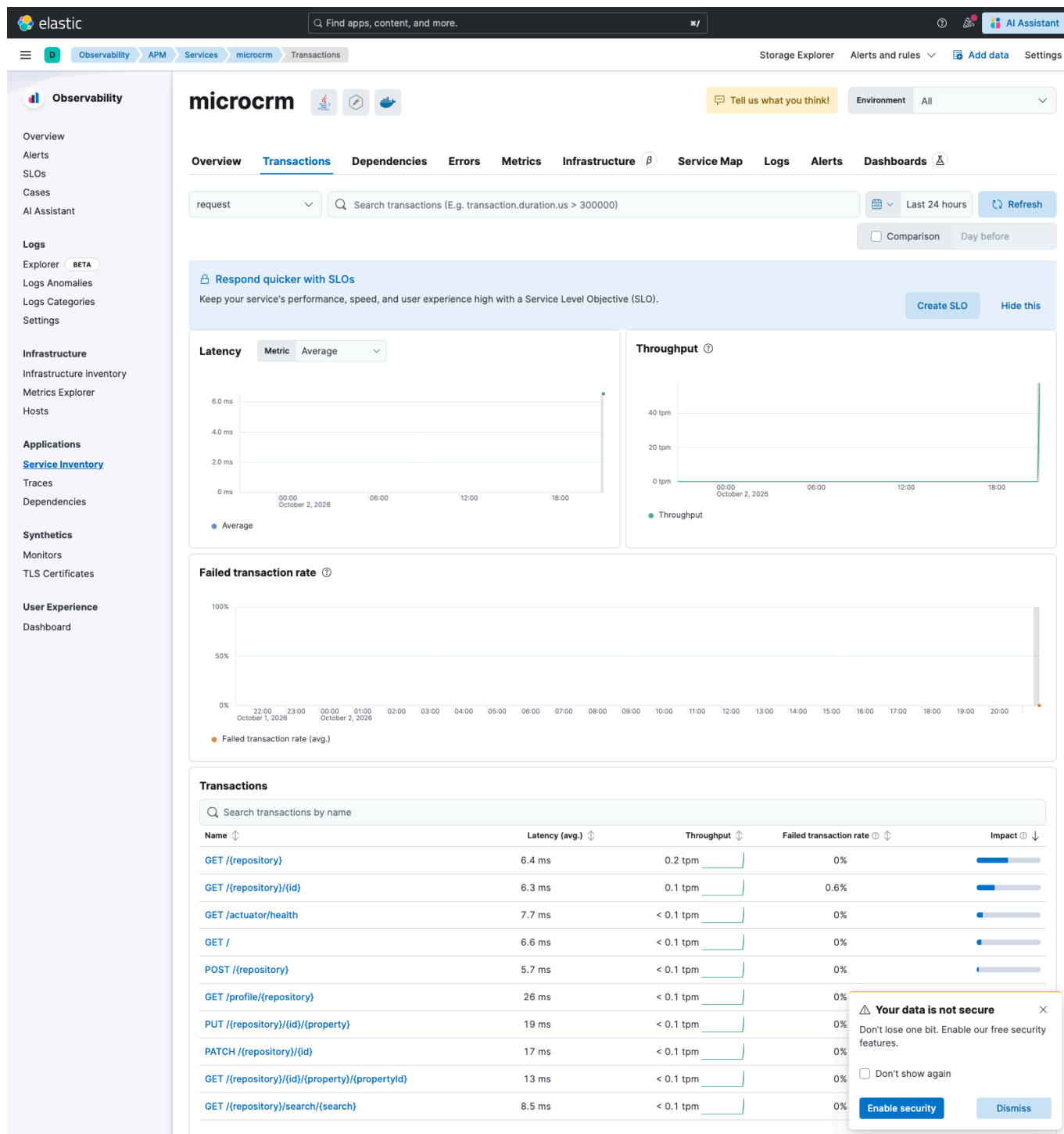
Ce rapport écrivait qu'il n'existait « aucune latence d'API ». C'est devenu faux **en partie seulement**, et la frontière mérite d'être tracée.

Depuis le commit `143adb7` (1^{er} octobre), le back embarque un agent OpenTelemetry qui trace chaque requête et l'envoie à Elastic APM, sans qu'une ligne du code Java ait changé. **Cette chaîne est écrite et éprouvée depuis le poste ; elle n'est pas déployée.** Les pods de staging et de production exécutent des images du 23 septembre, antérieures à l'agent : le 2 octobre, aucune trace ne sortait du cluster.

Pour ne pas en rester à une lecture de manifestes, l'image instrumentée a été lancée dans un conteneur local, et 303 requêtes lui ont été envoyées en 18 secondes :

Transaction	Requêtes	p50	p95	p99
GET <code>/repository</code>	200	4,20 ms	5,65 ms	8,60 ms
GET <code>/repository/{id}</code>	80	2,64 ms	3,41 ms	6,03 ms
POST <code>/repository</code>	20	3,01 ms	5,02 ms	24,25 ms

`{repository}` est la route générique de Spring Data REST : `/persons` et `/organizations` y sont confondus.



Ce qu'il faut y lire : l'étiquette d'environnement « verification-poste » sur la première capture. Elle dit que ces mesures viennent d'un essai lancé depuis le poste, pas de staging. Sur la seconde, toute l'activité tient dans la dernière heure : c'est un essai, pas un trafic.

⚠ Trois réserves sur ces chiffres. Ce sont les latences d'un conteneur local, sur un poste chargé, avec une base de démonstration presque vide : elles prouvent que la mesure existe, pas ce que vaut l'API de staging. La seconde capture agrège aussi les requêtes d'un autre essai local, celui des alertes (environnement `demo-alerting`) : ses chiffres diffèrent donc du tableau. Enfin l'agent a un coût, mesuré une seule fois : 408 Mio de mémoire contre 292 Mio, et un démarrage en 3,39 s contre 2,45 s. C'est un ordre de grandeur, à confronter à la limite de 768 Mio du pod avant tout déploiement.

Ce qui reste non mesuré : le CPU et la mémoire. Les traces disent ce que l'API fait de ses requêtes, pas ce que ses pods consomment. `kubectl top` répond toujours `Metrics API not available`, faute de `metrics-server` (§6.7).

5.4 Comportement sous déploiement et sous panne

Deux campagnes sur cluster ont mesuré ce qu'un utilisateur subit pendant les opérations, en interrogeant l'API en continu à travers l'Ingress.

Situation	Mesure
Rollout initial du back	<code>rollout status</code> en 0, ~10,5 s
Déploiement + rollback (180 s d'observation)	1 615 requêtes, 1 611 en 200 — 99,75 %
Fenêtre d'indisponibilité pendant un déploiement	≤ 0,20 s, un seul échantillon en erreur
Suppression du pod back	16,1 s d'indisponibilité totale (98 × 503 sur 350 requêtes)
Démarrage applicatif du back	6,7 s à 9,8 s selon l'exécution (±45 %)
Délai jusqu'à Ready	10 s à 15 s ; 2 tentatives de <code>startupProbe</code> sur 30

Ces chiffres disent deux choses distinctes, qu'il ne faut pas confondre.

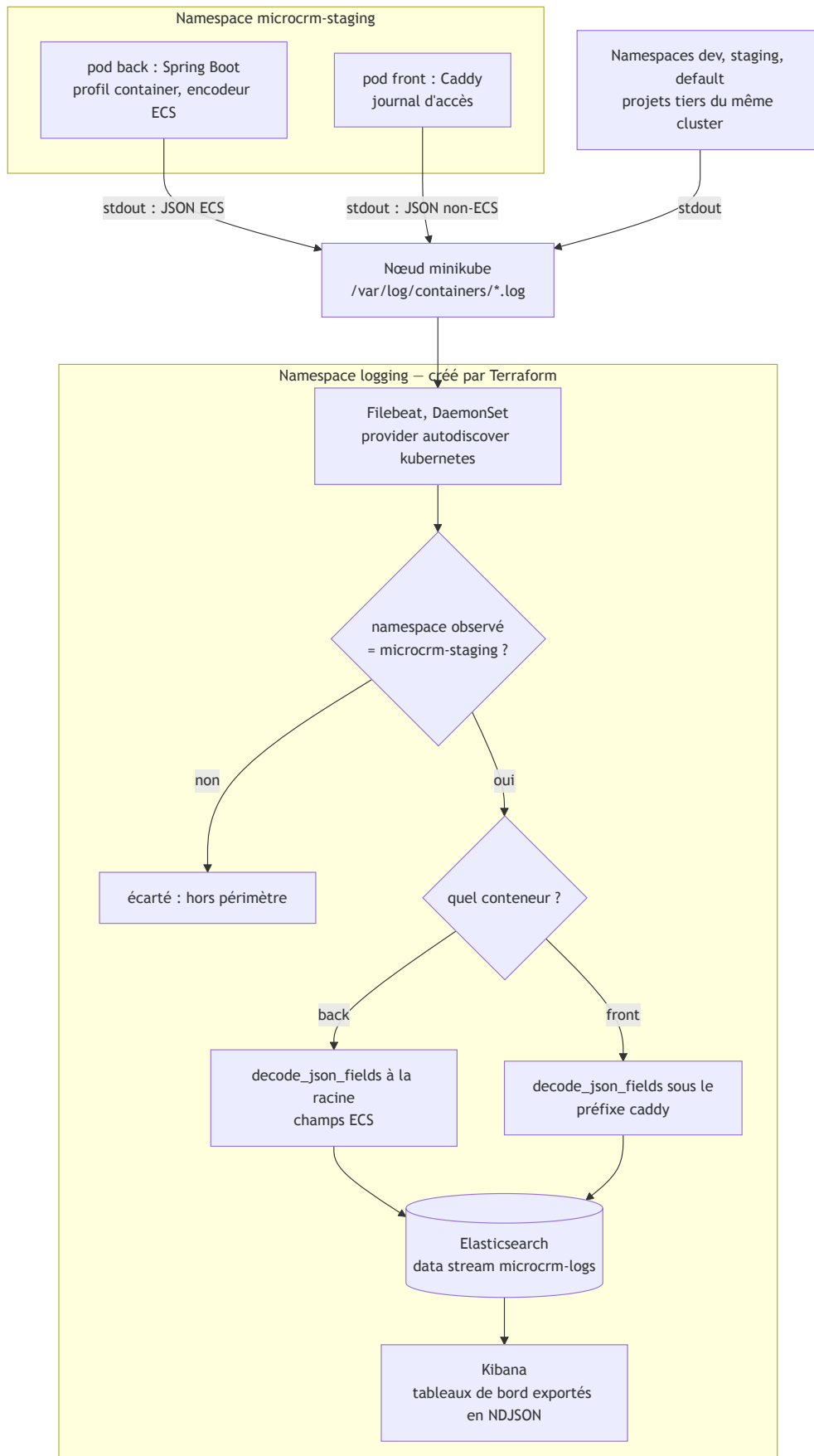
Un déploiement ne coupe pas. Le réglage `maxUnavailable: 0` impose que le nouveau pod soit prêt avant que l'ancien ne parte ; un déploiement vers une image inexistante n'a jamais touché le pod en service (`RESTARTS 0`, âge inchangé, API à 200 pendant toute la fenêtre). Le correctif de l'overlay éphémère n'a d'ailleurs rien amélioré sur la disponibilité — l'ancien mécanisme ne coupait pas non plus — il a corrigé la justesse de l'historique des révisions, donc la fiabilité du rollback.

Une perte de pod coupe seize secondes. C'est la conséquence directe du `replicas: 1` imposé par une base en mémoire : à un seul pod, il n'y a rien pour absorber la perte, et aucun `PodDisruptionBudget` ne protège d'une éviction. Le front, statique et à deux replicas en production, n'a pas été affecté. La levée de cette limite passe par la sortie de HSQLDB, pas par un réglage.

La variance du démarrage (6,7 s à 9,8 s sur une machine identique et non chargée) renforce le choix d'un `startupProbe` plutôt que d'un `initialDelaySeconds` fixe : si le délai bouge de moitié sans qu'aucune variable ne change, un délai figé serait le mauvais outil. Le budget actuel — 150 s — est très surdimensionné (2 tentatives consommées sur 30) ; son vrai prix est l'autre bout, une JVM réellement bloquée occupant un slot 150 s avant d'être abandonnée.

6. Supervision : tableaux de bord, indicateurs et alertes

6.1 Le flux des logs



Source versionnée : `docs/schemas/flux-logs.mmd`, reprise dans `ARCHITECTURE.md` §8.3.

Le chemin lui-même est banal — un pod écrit sur `stdout`, le runtime en fait un fichier sur le nœud, un agent le lit. **Les deux losanges sont l'intérêt du schéma**, parce qu'ils portent les deux décisions sans lesquelles la chaîne ne tient pas.

Le premier filtre est une question de périmètre, pas de volume. Ce cluster n'est pas dédié à MicroCRM : il héberge les namespaces `dev` et `staging` d'autres projets, et quatre pods dans `default`. Un Filebeat non filtré y ingérerait les logs de tiers. Le filtre est posé deux fois — sur ce que l'autodiscover *observe*, et en condition sur ce qu'il *traite* — pour que l'élargissement de l'un ne fasse pas tomber l'autre. Sa contrepartie est à connaître : **seul `microcrm-staging` est collecté, la production ne l'est pas.**

Le second embranchement évite une panne irréversible. Le back produit de l'ECS, décodé à la racine ; le front produit lui aussi du JSON, mais non-ECS, dont les champs portent les mêmes noms sans avoir la même forme. Un décodage uniforme faisait rejeter ses documents, et une collision de mapping ne se répare pas : une fois le champ typé dans l'index, aucun document contradictoire n'y entrera plus.

6.2 Ce qui a été vérifié

Vérification	Résultat
Rollout Elasticsearch / Kibana / Filebeat	les trois <code>Running</code>
Documents indexés (16 août 2026)	50 , dont 36 du conteneur back
Documents indexés (2 octobre 2026)	92 175 , un seul namespace
Provenance	100 % <code>microcrm-staging</code> — aucun projet voisin collecté
Champs ECS décodés	<code>log.level</code> , <code>log.logger</code> , <code>service.name</code> , <code>process.thread.name</code>
Métadonnées Kubernetes	<code>kubernetes.pod.name</code> , <code>kubernetes.namespace</code> , <code>container.image.name</code>
Réimportation des tableaux de bord	cinq fichiers, <code>success: true</code> sur chacun (§6.4)
APM Server (2 octobre 2026)	1/1 <code>Running</code> , version 8.19.7 ; aucune trace venue du cluster (§5.3)

Le livrable n'est pas « des écrans dans Kibana », ce sont des fichiers. Les tableaux de bord sont exportés en NDJSON et les règles d'alerte en JSON, tous versionnés dans le dépôt. Un écran ou une règle qui n'existe que dans une instance disparaît avec elle — et celle-ci tourne sur un poste de

développement. Chaque panneau chiffré a été confronté à la même agrégation jouée directement contre Elasticsearch : mêmes chiffres des deux côtés (`k8s/elk/dashboards/README.md`).

Le dimensionnement de la stack a été confronté à une charge réelle, ce qui n'est pas la même chose qu'un plan validé. Pendant le rollout de Kibana, le quota du namespace affichait `limits.memory 5376Mi/6Gi`, `requests.memory 3200Mi/4Gi`, pods 4/12 — **exactement les valeurs calculées dans le `terraform.tfvars`, au mégaoctet près**. La règle qui les gouverne mérite d'être retenue : la limite mémoire vaut le double du tas, parce qu'un conteneur JVM dont la limite égale le tas est tué par l'OOM killer au premier pic hors-tas. C'est la cause la plus banale d'une stack ELK qui « ne démarre pas » sans message utile.

6.3 Les indicateurs retenus, et pourquoi

Un indicateur n'a de valeur que si l'on sait d'où il vient et ce qu'il ne dit pas. Les douze ci-dessous sont ceux que les tableaux de bord affichent.

Domaine	Indicateur (KPI)	Source	Pourquoi celui-là	Sa limite
Livraison	Les quatre indicateurs DORA	API GitLab, <code>collect_dora.py</code>	Mesurent le résultat de la chaîne, vitesse et stabilité ensemble	Très peu d'observations (§2.3)
Disponibilité	Sondes de Kubernetes servies par le front	journal d'accès de Caddy	Un battement régulier, 18 requêtes par minute : son silence est une panne	Ne distingue pas front arrêté et collecte arrêtée
Disponibilité	Démarrages du back	ligne <code>Started MicroCRMApplication</code>	Un back qui redémarre en boucle se voit là	Ce sont des démarrages, pas le compteur de redémarrages
Disponibilité	Requêtes terminées en erreur	logs du back, traces APM	Chaque occurrence est une réponse 500 servie à un utilisateur	Côté traces : pas encore en service

Domaine	Indicateur (KPI)	Source	Pourquoi celui-là	Sa limite
Performance	Latence du front, p50 / p95 / p99	<code>caddy.duration</code>	Un percentile montre les utilisateurs lents qu'une moyenne cache	Latence du serveur de fichiers, pas de l'API
Performance	Latence et débit de l'API, par route	traces OpenTelemetry	Dit quelle route ralentit, pas seulement « c'est lent »	Mesuré sur un conteneur local seulement (§5.3)
Performance	Volume de logs par conteneur	<code>microcrm-logs</code>	Révèle un pic d'activité ou un service devenu muet	Compte des lignes, pas des requêtes
Sécurité	Constats CRITICAL et HIGH ouverts	rapports Trivy, <code>microcrm-security</code>	Ce sont les deux sévérités qui arrêtent le pipeline	Vaut pour l'image publiée, pas pour le code
Sécurité	Constats ouverts dans le temps	idem	Montre si la dette de sécurité monte ou descend	Historique rejoué, pas vécu (§6.4)
Sécurité	Exceptions et jours avant échéance	<code>.trivyignore.yaml</code>	Une exception oubliée devient une faille acceptée pour toujours	N'inclut pas les exceptions Dependency-Check
Sécurité	Chemins sensibles demandés au front	<code>caddy.request.uri</code>	<code>/.env</code> , <code>/.git</code> : la signature d'un balayage automatique	Signale une tentative, pas une fuite
Alertes	Déclenchements par famille et par règle	index <code>microcrm-alerts</code>	Dit ce qui a sonné, quand, et si c'est rétabli	Un écran vide ne veut pas dire que tout va bien

6.4 Les tableaux de bord

Cinq tableaux de bord sont versionnés dans `k8s/elk/dashboards/`. Tous ont été réimportés sur une instance d'où leurs objets avaient été supprimés.

Tableau de bord	Fichier	Objets	Ce qu'il suit	Capture
Supervision	<code>microcrm.ndjson</code>	8	performance et erreurs : volume, latence, statuts	§6.4
Sécurité	<code>securite.ndjson</code>	22	vulnérabilités, exceptions, erreurs	§6.4
Disponibilité	<code>disponibilite.ndjson</code>	18	présence, taux de réussite, démarrages, API	§6.4
Suivi des alertes	<code>alertes.ndjson</code>	8	déclenchements et rétablissements des huit règles	§6.5
Métriques DORA	<code>dora.ndjson</code>	13	les quatre indicateurs de livraison	§2.3

Les captures ci-dessous datent toutes du 2 octobre 2026. Elles remplacent celles du 19 septembre, dont la capture DORA montrait l'état d'avant le déblocage de la chaîne.

Comment lire ce tableau de bord

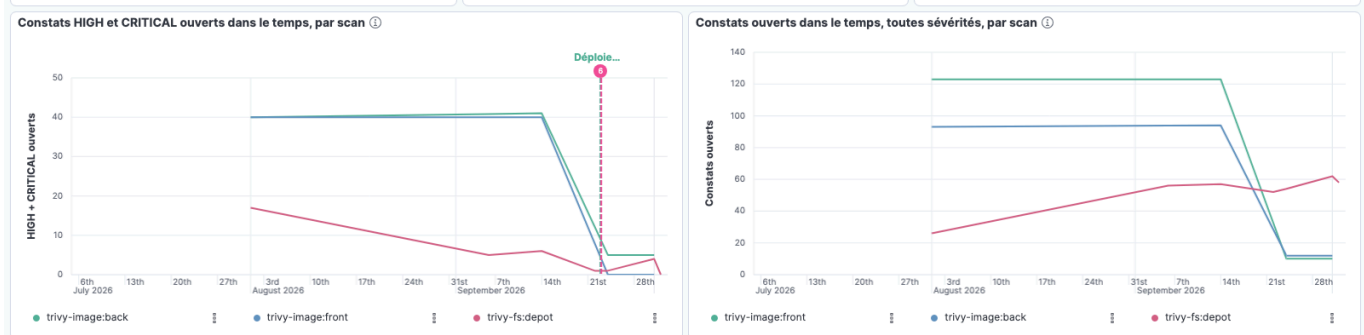
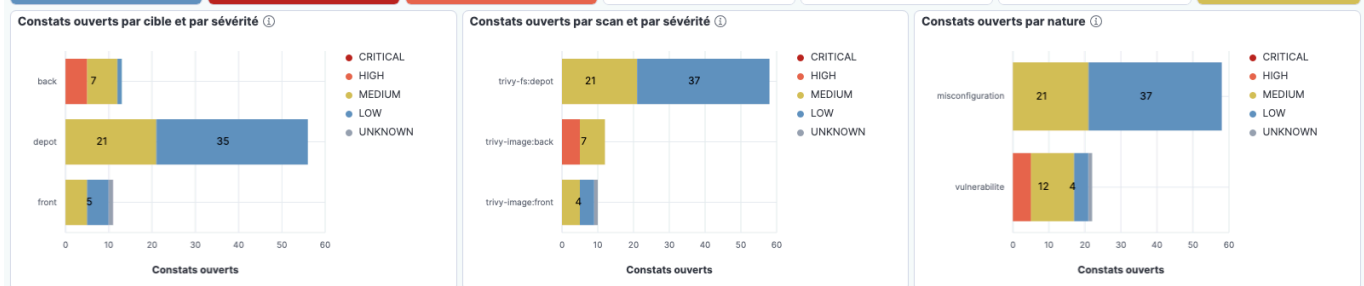
D'où viennent ces chiffres. `scripts/ci/collect_security.py` lit les rapports JSON de Trivy (dépôt et images) et les exceptions de `.trivyignore.yaml`, puis les envoie dans l'index `microcrm-security`. Rien n'est calculé par Kibana au-delà d'un comptage.

État courant et historique. Les tuiles, les trois histogrammes et la table des constats ne montrent que le **dernier scan de chaque source** (courant : `true`) : dépôt, image du back, image du front. Les deux courbes, elles, gardent tous les scans.

Zéro mesuré ou pas de mesure. Lire d'abord la tuile « Scans pris en compte ». Si elle affiche `3`, un `0` dans une tuile voisine est une mesure : les scans ont tourné et n'ont rien trouvé. Si elle affiche `0`, aucun scan n'a été collecté sur la période, et les `0` voisins ne sont pas une bonne nouvelle : c'est une absence de donnée.

Ouvert ou excepté. Un constat couvert par une entrée de `.trivyignore.yaml` (même identifiant et même fichier, échéance non dépassée) est compté à part comme **excepté**. La table du bas donne chaque exception avec son échéance.

Limites. Les scans antérieurs à aujourd'hui ont été rejoués sur d'anciens commits avec la base de vulnérabilités d'aujourd'hui : la courbe montre ce que ces versions contiennent de vulnérable au regard de ce qu'on sait maintenant, pas ce que le pipeline voyait à l'époque. Les panneaux HTTP ne voient que le **front** en staging : le back ne journalise pas ses réponses.



Vulnérabilités et constats HIGH / CRITICAL ouverts — dernier scan de chaque source 

5 documents

  @timestamp	 severity	 identifiant	 paquet	 version	 version_corrigee	 composant	 source	 cible	 commit
 Oct 1, 2026 @ 19:05:51.000	HIGH	CVE-2026-91777	com.fasterxml.jacks	2.21.4	2.21.7, 2.18.11, 2.22.3	back	trivy-image:back	Java	Sbf1d6a2
 Oct 1, 2026 @ 19:05:51.000	HIGH	CVE-2026-91776	com.fasterxml.jacks	2.21.4	2.18.11, 2.21.7, 2.22.3	back	trivy-image:back	Java	Sbf1d6a2
 Oct 1, 2026 @ 19:05:51.000	HIGH	CVE-2026-89425	com.fasterxml.jacks	2.21.4	2.21.7, 2.22.3, 2.18.11	back	trivy-image:back	Java	Sbf1d6a2
 Oct 1, 2026 @ 19:05:51.000	HIGH	CVE-2026-89487	com.fasterxml.jacks	2.21.4	2.18.11, 2.21.7, 2.22.3	back	trivy-image:back	Java	Sbf1d6a2
 Oct 1, 2026 @ 19:05:51.000	HIGH	CVE-2026-68497	com.fasterxml.jacks	2.21.4	2.18.10, 2.21.6, 2.22.2	back	trivy-image:back	Java	Sbf1d6a2

Exceptions assumées et leur échéance 

7 documents

  @timestamp	 identifiant	 chemin	 echeance	 jours_restants	 expire	 constats_couvres	 justification
 Oct 2, 2026 @ 21:32:14.000	KSV-0056	.gitlab/agents/microcrm/rbac.yaml	Dec 31, 2026 @ 01:00:00.000	90	false	3	Droits d'écriture sur les ressources networking accordés à l'agent ...
 Oct 2, 2026 @ 21:32:14.000	KSV-0041	.gitlab/agents/microcrm/rbac.yaml	Dec 31, 2026 @ 01:00:00.000	90	false	1	Gestion des Secrets accordée à l'agent GitLab, et la règle a ...
 Oct 2, 2026 @ 21:32:14.000	KSV-0014	k8s/elk/elasticsearch-deployment.yaml	Dec 31, 2026 @ 01:00:00.000	90	false	1	Écart assumé et déjà commenté dans le manifeste. L'entrypoint...
 Oct 2, 2026 @ 21:32:14.000	KSV-0014	k8s/overlays/production/back-resources-patch.yaml	Dec 31, 2026 @ 01:00:00.000	90	false	1	Faux positif de méthode, pas d'appréciation. Ce fichier est un patch ...
 Oct 2, 2026 @ 21:32:14.000	KSV-0118	k8s/overlays/production/b	Dec 31, 2026 @	90	false	2	Même cause que l'entrée



Ce qu'il faut y lire, de haut en bas. La tuile « 3 scans » garantit que les zéros voisins sont mesurés et non vides. La tuile rouge affiche 0 CRITICAL, la tuile orange 5 HIGH. La courbe de gauche montre la chute des constats graves sur les deux images. La première table nomme les 5 CVE et la version qui les corrige. La seconde liste les exceptions et leurs 90 jours restants.

Analyse. Trois enseignements.

- **Les constats graves ont été divisés par huit sur l'image du back** (40 puis 41, puis 5), et ramenés à zéro sur celle du front (40, puis 0). La bascule date du 19 septembre : les scans Trivy sont devenus bloquants, et le même jour Spring Boot est passé de 3.2.5 à 3.5.16 et Caddy a été recompilé pour fermer les CVE (commit `df1634f`).
- **Le dépôt est passé de 17 constats graves à 0**, les défauts de configuration restants étant soit corrigés, soit couverts par une exception datée.
- **Ce qui reste ouvert est de faible sévérité** : 33 MEDIUM et 41 LOW, dont 58 défauts de configuration. Ils ne bloquent pas la livraison et forment la dette à résorber (\$7.4).

⚠ **Trois réserves, sans lesquelles ces courbes se lisent mal.**

- **L'historique est rejoué, il n'a pas été vécu.** Aucun scan n'avait été collecté avant le 2 octobre. Huit anciens commits ont été scannés ce jour-là, avec la base de vulnérabilités de ce jour-là, et datés de leur commit. La courbe dit « ce que ces versions contiennent de vulnérable au regard de ce qu'on sait aujourd'hui », pas « ce que le pipeline voyait à l'époque ».
- **Les trois panneaux du bas sont vides par construction.** Le front répond 200 à tout chemin, et le back, seul à pouvoir répondre 401 ou 403, ne journalise pas ses accès. Leur titre le dit. L'application n'a d'ailleurs pas d'authentification.
- **Dependency-Check n'y figure pas.** Le collecteur sait lire son rapport — testé sur un rapport fabriqué — mais aucun rapport réel n'a été collecté. Et le collecteur n'a pas encore été exécuté sur les rapports d'un pipeline.

Disponibilité

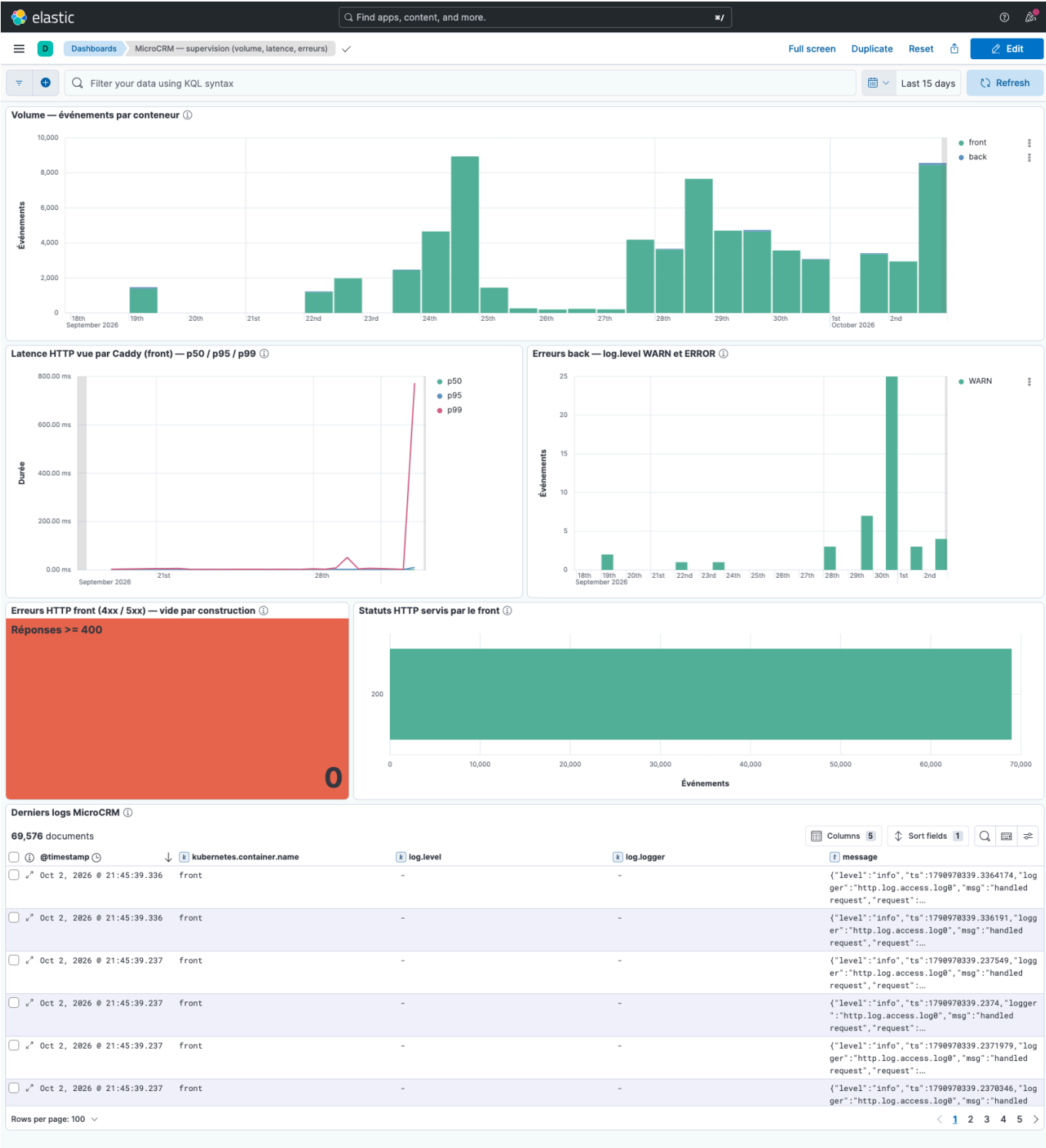


Ce qu'il faut y lire : la grande courbe verte est le battement de cœur du front. Ses trous sont les périodes où le poste était éteint. Les traits verticaux sont des annotations automatiques : un démarrage du back, un déploiement, un rollback.

Analyse. Ce tableau de bord est le plus facile à sur-lire, et il faut le dire nettement : **il ne mesure pas un taux de disponibilité, il montre une présence de logs.**

- **Le 100 % de réussite est vrai et ne prouve presque rien.** Le front répond 200 à tout, et 94 % de ses requêtes sont des sondes. Ce taux dit que Caddy répond.
- **Sur les 360 heures de la fenêtre, 189 portent au moins une sonde,** soit 52,5 %. Ce n'est pas un taux de disponibilité : le cluster est un minikube de poste, éteint la nuit, et un trou ne distingue pas « front arrêté » de « collecte arrêtée ».
- **Onze démarrages du back en quinze jours,** dont trois le 2 octobre. Ces trois-là s'expliquent par la relance du cluster et les essais d'alerte, pas par des pannes — c'est ce que les annotations permettent de vérifier d'un coup d'œil.
- **Les panneaux APM ne décrivent pas l'application déployée** : leurs transactions viennent des deux essais locaux du 2 octobre.
- **La production n'apparaît pas.** Le panneau « par environnement » n'a qu'une série, et il existe pour que cette absence se voie.

Performance



Ce qu'il faut y lire : la courbe de latence, en haut à gauche, reste plate pendant quinze jours puis monte brutalement à la dernière heure. Le panneau voisin compte les avertissements du back, jour par jour.

Analyse.

- **La pointe de latence du 2 octobre est provoquée, pas subie.** C'est l'essai de l'alerte de performance : le front de staging a été privé de CPU, sous charge, pour faire sonner la règle (\$6.5). Le reste de la fenêtre est plat, conforme au p95 médian de 1,54 ms.

- **Le back n'a journalisé aucune erreur.** Sur 90 jours : 52 lignes `WARN` , 0 `ERROR` . Les avertissements du back sont ceux qu'il écrit en démarrant, auxquels s'ajoutent ceux de l'essai d'alerte du 2 octobre. La cause du pic de 25 lignes sur une seule journée n'a pas été établie ligne à ligne.
- **Le volume suit les heures d'allumage du poste**, pas un trafic d'utilisateurs : il n'y a pas d'Ingress exposé, donc pas de visiteur.

6.5 Les alertes

Jusqu'au 2 octobre, la supervision se consultait : rien ne prévenait. Huit règles d'alerte Kibana couvrent désormais les trois familles attendues. Ce sont des fichiers (`k8s/elk/alerting/rules/`), installés par `scripts/monitoring/install_alerting.py`, et non des réglages faits à la souris. Toutes s'évaluent chaque minute.

Famille	Règle	Ce qu'elle surveille	Seuil	Pourquoi ce seuil
Disponibilité	<code>dispo-front-muet</code>	le front de staging ne sert plus rien	moins de 1 requête en 3 min	54 sondes attendues en 3 min : zéro n'est jamais un creux, c'est une panne
Disponibilité	<code>dispo-back-redemarrages</code>	le back redémarre en boucle	2 démarrages ou plus en 15 min	12 démarrages en 47 jours, jamais plus d'un par quart d'heure sauf une fois
Disponibilité	<code>dispo-back-echecs-requetes</code>	des requêtes du back finissent en exception	1 ou plus en 5 min	aucune ligne de ce genre en 47 jours
Disponibilité	<code>dispo-api-5xx</code>	l'API répond en erreur 5xx (traces)	1 ou plus en 5 min	0 sur les 401 transactions relevées avant l'essai
Performance	<code>perf-front-p95</code>	le front ralentit	p95 > 100 ms sur 20 requêtes	65 fois le p95 médian de 1,54 ms ; le plancher écarte les tranches trop petites
Performance	<code>perf-api-p95</code>	l'API ralentit (traces)	p95 > 250 ms sur 20 transactions	2,5 fois le maximum observé (97 ms) ; échantillon petit, seuil à recalculer

Famille	Règle	Ce qu'elle surveille	Seuil	Pourquoi ce seuil
Sécurité	secu-front-chemins-sensibles	quelqu'un demande /.env, /.git, /wp-login.php...	1 requête ou plus en 5 min	sur 68 493 requêtes en 30 jours, le seul chemin demandé était /
Sécurité	secu-api-rafale-4xx	énumération d'identifiants sur l'API (traces)	20 réponses 4xx ou plus en 5 min	trafic normal : 0 ; une énumération en produit 40 à 64

elastic

Find apps, content, and more.

Stack Management

Rules

Management

Ingest

Data

Alerts and Insights

Kibana

Stack

Rules

Detect conditions using rules.

Rules Logs

Search

Rule state 0 Type 0 Action type 0 Last response 0 Tags 0 Refresh

Succeeded: 8 Failed: 0 Warning: 0 Updated a few seconds ago 5 m

8 rules

Name	Last run	Notify	Inter...	Duration	P50	Success rati...	Last response	State
MicroCRM - disponibilité - back de staging qui redémarre en boucle Elasticsearch query	Oct 2, 2026 21:56:36pm a few seconds ago		1 min	00:00	00:01	100%	Succeeded	Enabled
MicroCRM - disponibilité - front de staging muet Elasticsearch query	Oct 2, 2026 21:56:36pm a few seconds ago		1 min	00:00	00:00	100%	Succeeded	Enabled
MicroCRM - disponibilité - requêtes du back de staging terminées en exception Elasticsearch query	Oct 2, 2026 21:56:33pm a few seconds ago		1 min	00:01	00:01	100%	Succeeded	Enabled
MicroCRM - disponibilité - réponses 5xx de l'API (traces APM) Elasticsearch query	Oct 2, 2026 21:56:30pm a few seconds ago		1 min	00:01	00:01	100%	Succeeded	Enabled
MicroCRM - performance - latence p95 de l'API (traces APM) Elasticsearch query	Oct 2, 2026 21:56:39pm a few seconds ago		1 min	00:00	00:00	100%	Succeeded	Enabled
MicroCRM - performance - latence p95 du front de staging Elasticsearch query	Oct 2, 2026 21:56:42pm a few seconds ago		1 min	00:00	00:00	100%	Succeeded	Enabled
MicroCRM - sécurité - chemins sensibles demandés au front de staging Elasticsearch query	Oct 2, 2026 21:56:48pm a few seconds ago		1 min	00:00	00:00	100%	Succeeded	Enabled
MicroCRM - sécurité - rafale de réponses 4xx de l'API (traces APM) Elasticsearch query	Oct 2, 2026 21:56:42pm a few seconds ago		1 min	00:00	00:00	100%	Succeeded	Enabled

Rows per page: 10 1

Your data is not secure

Don't lose one bit. Enable our free security features.

Don't show again

Enable security Dismiss

Ce qu'il faut y lire : les huit règles sont actives et leur dernière évaluation a réussi. « Succeeded » veut dire que la règle s'est exécutée sans erreur, pas qu'une alerte a sonné.

Preuve de déclenchement. Une règle qui n'a jamais sonné ne prouve rien. Le 2 octobre, entre 21 h 37 et 21 h 54 (heure de Paris), chacune des huit a été déclenchée volontairement, puis s'est rétablie seule.

Règle	Provoquée par	Déclenchée	Valeur relevée	Rétablie
dispo-back- echecs-requetes	une requête invalide sur le back de staging	21:37:35	1 exception	21:40:35
dispo-api-5xx	la même requête, sur un back instrumenté local	21:37:35	1 réponse 5xx	21:40:35
dispo-back- redemarrages	deux redémarrages du back de staging	21:38:38	3 démarrages	21:53:39
secu-api-rafale- 4xx	60 demandes d'identifiants inexistants, back local	21:38:47	64 réponses	21:43:44
secu-front- chemins-sensibles	8 requêtes /.env, /.git/config... au front	21:38:50	8 requêtes	21:43:50
perf-api-p95	back local bridé à 0,05 CPU	21:40:41	269,2 ms	21:44:41
perf-front-p95	front de staging bridé à 10 m CPU, sous charge	21:45:44	909,1 ms	21:51:45
dispo-front-muet	front de staging ramené à 0 replica	21:50:39	0 requête	21:51:39

elastic

Find apps, content, and more.

MicroCRM — suivi des alertes (disponibilité, performance, sécurité)

Full screenDuplicateResetEdit

Filter your data using KQL syntax

Last 1 hourRefresh

Comment lire le suivi des alertes

Chaque ligne de ce tableau de bord est un changement d'état d'une règle d'alerte Kibana, écrit dans l'index `microcrm-alerts` par le connecteur `microcrm-alerts-index` : `declenchee` quand la condition devient vraie, `retablie` quand elle cesse de l'être. Une alerte qui dure dix minutes produit donc deux lignes, pas dix.

Les règles, leurs requêtes et leurs seuils sont des fichiers : `k8s/elk/alerting/rules/`, expliqués dans `k8s/elk/alerting/README.md`.

Un écran vide ne veut pas dire que tout va bien. Il veut dire qu'aucune règle n'a changé d'état sur la période — ce qui est aussi ce qu'on verrait si Kibana était arrêté ou si les règles avaient disparu. L'état des règles elles-mêmes se lit dans *Stack Management > Rules*, ou par `scripts/monitoring/install_alerting.py --etat`.

Ce que ces alertes ne voient pas : la production (Filebeat ne collecte que `microcrm-staging`), et l'API tant que le back déployé n'envoie pas de traces — les règles « traces APM » ne surveillent aujourd'hui que les environnements qui en émettent.

Déclenchements sur la période

Déclenchements

8

Chronologie des déclenchements, par famille

Famille	21:35	21:40	21:45	21:50
disponibilité	2	1	0	1
securite	0	0	0	0
performance	0	1	1	0

Déclenchements par famille

Famille	Déclenchements
disponibilité	4
performance	2
securite	2

Changements d'état par règle

Règle	declenchee	retablie
dispo-api-5xx	1	1
dispo-back-echecs-requetes	1	1
dispo-back-redemarrages	1	1
dispo-front-muet	1	1
perf-api-p95	1	1
perf-front-p95	1	1
secu-api-rafale-4xx	1	1
secu-front-chemins-sensibles	1	1

Journal des alertes

16 documents

@timestamp

famille

severite

regle_id

statut

valeur

unite

seuil

detail

✓ Oct 2, 2026 @ 21:53:39.075	disponibilite	haute	dispo-back-redemarrages	retablie	- démarrages		2 démarrages ou plus en 15 min	la condition n'est plus remplie
✓ Oct 2, 2026 @ 21:51:45.017	performance	moyenne	perf-front-p95	retablie	- ms		p95 > 100 ms sur 5 min (20 requêtes au moins)	la condition n'est plus remplie
✓ Oct 2, 2026 @ 21:51:39.033	disponibilite	critique	dispo-front-muet	retablie	- requêtes		moins de 1 requête servie en 3 min	la condition n'est plus remplie
✓ Oct 2, 2026 @ 21:50:39.023	disponibilite	critique	dispo-front-muet	declenchee	0 requêtes		moins de 1 requête servie en 3 min	aucune ligne du journal d'accès Caddy : front arrêté, ou ...
✓ Oct 2, 2026 @ 21:45:44.886	performance	moyenne	perf-front-p95	declenchee	989.1 ms		p95 > 100 ms sur 5 min (20 requêtes au moins)	p95 calculé sur 989 requêtes
✓ Oct 2, 2026 @ 21:44:41.862	performance	moyenne	perf-api-p95	retablie	- ms		p95 > 250 ms sur 5 min (20 transactions au moins), par ...	la condition n'est plus remplie
✓ Oct 2, 2026 @ 21:43:50.851	securite	haute	secu-front-chemins-sensibles	retablie	- requêtes		1 requête ou plus vers un chemin sensible en 5 min	la condition n'est plus remplie
✓ Oct 2, 2026 @ 21:43:44.845	securite	moyenne	secu-api-rafale-4xx	retablie	- réponses 4xx		20 réponses 4xx ou plus en 5 min, par environnement	la condition n'est plus remplie
✓ Oct 2, 2026 @ 21:40:41.248	performance	moyenne	perf-api-p95	declenchee	269.2 ms		p95 > 250 ms sur 5 min (20 transactions au moins), par ...	environnement demo-alerting, p95 calculé sur 553 transactions
✓ Oct 2, 2026 @ 21:40:35.239	disponibilite	haute	dispo-api-5xx	retablie	- réponses 5xx		1 réponse 5xx ou plus en 5 min, par environnement	la condition n'est plus remplie
✓ Oct 2, 2026 @ 21:40:35.233	disponibilite	haute	dispo-back-echecs-requetes	retablie	- exceptions		1 exception journalisée ou plus en 5 min	la condition n'est plus remplie

Rows per page: 100

1

Your data is not secure

Don't lose one bit. Enable our free security features.

Don't show again

Enable securityDismiss

Ce qu'il faut y lire : chaque règle apparaît deux fois, « déclenchée » puis « rétablie ». Le journal du bas donne la valeur qui a fait sonner — 909,1 ms pour la latence du front, 269,2 ms pour celle de l'API.

Un défaut applicatif découvert en testant. La requête `GET /persons/abc` renvoie une erreur 500 au lieu d'un 400 : l'application traite une saisie invalide comme une panne. C'est un défaut à corriger côté code, que l'alerte a rendu visible.

⚠ **Ce qu'il ne faut pas sur-lire.**

- **Aucune notification ne sort de Kibana.** Une alerte écrit une ligne dans un index et une ligne dans le journal de Kibana : il faut ouvrir le tableau de bord pour la voir. Les connecteurs courriel, Slack et webhook exigent une licence payante.
- **Les trois règles fondées sur les traces sont prouvées sur un conteneur local, pas sur les pods déployés.** Elles ne surveilleront staging et production qu'après le déploiement d'une image instrumentée.
- **Les cinq autres ne voient que staging,** seul environnement collecté.
- **L'alerting ne se surveille pas lui-même.** Si Kibana tombe, aucune règle ne s'évalue, et un écran vide se lit alors comme « tout va bien ».
- **Un back arrêté sans redémarrer n'est détecté par aucune règle** aujourd'hui.
- `dispo-back-redemarrages` affiche 3 et non 2 parce que le démarrage du cluster avait lui-même lancé le back douze minutes plus tôt.

6.6 Ce que les données apprennent : points de vigilance et actions

Constat tiré des tableaux de bord	Ce qu'il signifie	Action proposée (§7.4)
Aucun déploiement depuis le 23 septembre	Les correctifs de sécurité s'accumulent sans être mis en service	Remettre <code>develop</code> au vert, publier la 1.0.1
5 CVE hautes sur l'image publiée du back	L'image de référence au registry est connue vulnérable	Publier une image corrigée, supprimer l'ancienne
Une seule série dans « présence par environnement »	La production n'est pas supervisée	Étendre la collecte à la production
52,5 % des heures seulement portent des sondes	La disponibilité n'est pas mesurable sur un poste éteint la nuit	Une sonde externe, sur un cluster permanent
Les panneaux 401/403 et 4xx sont vides par construction	La supervision de sécurité ne voit pas les accès à l'API	Déployer le back instrumenté

Constat tiré des tableaux de bord	Ce qu'il signifie	Action proposée (§7.4)
16,1 s de coupure à la perte d'un pod	Un seul replica, imposé par la base en mémoire	PostgreSQL, puis deux replicas
GET /persons/abc répond 500	Une saisie invalide est comptée comme une panne	Corriger la conversion d'identifiant dans l'application
Mémoire avec agent : 408 Mio pour une limite de 768 Mio	La marge du pod est réduite, sur une seule mesure	metrics-server, puis recalage des ressources

6.7 Ce que la supervision ne fait pas

Volet	État au 2 octobre 2026
Sondes de santé (Actuator)	fait, vérifiées sous kubelet
Centralisation des logs	fait, pour staging seulement
Tableaux de bord versionnés	fait : cinq fichiers
Alerting	fait : 8 règles déclenchées et rétablies ; aucune notification hors de Kibana
Latence et débit de l'API	écrit et éprouvé depuis le poste, non déployé (§5.3)
Métriques CPU et mémoire	absentes — pas de metrics-server, métriques JVM de l'agent coupées
Supervision de la production	absente — Filebeat ne collecte que microcrm-staging
Rétention des logs	absente — sans ILM, les données s'accumulent jusqu'au PVC de 5 Gio
Sécurité d'Elasticsearch	désactivée (xpack.security.enabled: false)
Exposition de Kibana	aucun Ingress ; accès par port-forward uniquement
Automatisation	aucun job de CI ne déploie, ne teste ni n'alimente cette stack

L'absence de métriques de ressources a une conséquence directe qu'il faut nommer : **il manque jusqu'au signal sur lequel un autoscaler déciderait**, et les valeurs de ressources des manifestes restent des estimations — `kubectl top` répond `Metrics API not available`. Aucun `OOMKill` n'a été observé, ce qui est un indice, pas une mesure.

La sécurité désactivée d'Elasticsearch est assumée pour une stack locale de démonstration ; elle serait inacceptable ailleurs, et c'est la première chose à reprendre si cette stack devait sortir du poste. Les captures du §6.5 en portent la trace : Kibana y affiche lui-même « Your data is not secure ». C'est aussi pourquoi il n'y a pas d'Ingress : exposer une console d'administration sans authentification serait cohérent avec le point précédent, c'est-à-dire une mauvaise idée.

7. Gains obtenus et recommandations d'amélioration continue

7.1 Les indicateurs, avant et après

Chaque ligne ci-dessous est un état antérieur réel du dépôt, corrigé et vérifié — pas une amélioration théorique. Le premier tableau compare les indicateurs mesurés ; le second, les propriétés de la chaîne.

Indicateur	Avant	Après	Réserve
Déploiements réussis depuis la CI	0 sur 7 tentatives (jusqu'au 21 septembre)	5 sur 9 (fenêtre de 30 jours)	tous sur deux journées
Fréquence de déploiement	0,0 par jour	0,1667 par jour	retombera à 0 sans nouveau déploiement
Délai de mise en production	non mesurable	1,38 h (médiane)	contient l'attente du clic
Temps de rétablissement	non mesurable	2,14 h (médiane)	2 observations
Taux d'échec des changements	100 %	66,67 %	9 tentatives
Contrôles non bloquants	6 jobs en <code>allow_failure</code> , Trivy en <code>--exit-code 0</code>	1 seul contrôle non bloquant (<code>k6-load</code>)	—
Dépendances Java analysées	0	82	non commité
Constats graves, image du back	40 (commit du 1 ^{er} août)	5 sur l'image publiée ; 0 sur l'image 1.0.1 construite localement	historique rejoué ; image 1.0.1 non publiée
Constats graves, image du front	40	0	historique rejoué
Constats graves, dépôt	17	0	historique rejoué

Indicateur	Avant	Après	Réserve
Couverture des branches du back	62,5 %	100 %	—
Tests du front	73	112	exécution locale
Assertions sur les scripts	266	430	non engagé
Assertions sur les manifestes	108	151	non engagé
Latence de l'API	aucune mesure	p50, p95, p99 par route	conteneur local, non déployé
Règles d'alerte	0	8, toutes déclenchées puis rétablies	sans notification externe
Tableaux de bord versionnés	0	5 (69 objets)	—
Contexte de build du front	1 195 Mo envoyés au démon Docker	0,6 Mo	—
Contexte de build du back	51 Mo	0,1 Mo	—

Sujet	Avant	Après
Images d'outillage du pipeline	9 images flottantes ou en <code>latest</code>	toutes figées, déclarées une seule fois
Chaîne de compilation du back	trois JDK pour un même artefact (17 / 21 / 21)	une seule version, alignée Dockerfile ↔ pipeline
URL de l'API du front	compilée dans le bundle → <code>localhost:8080</code> en prod	lue au démarrage depuis <code>/config.json</code> — une image pour tous les environnements
Utilisateur des conteneurs	le front tournait en <code>root</code>	UID 1000 des deux côtés, aligné sur les manifestes
Port déclaré par le back	<code>EXPOSE 4200</code> pour une application sur <code>8080</code>	corrigé
Création d'un environnement	<code>kubectl create namespace</code> tapé par quelqu'un	décrit en Terraform, avec quota, limites et politiques

Sujet	Avant	Après
Prérequis du poste	deux phrases dans deux documents	playbook Ansible idempotent (<code>ok=23</code> <code>changed=0</code>)
Capacité de retour arrière	<code>rollback-production</code> ramenait un <i>placeholder</i> inexistant	ramène la version précédente réellement déployée, vérifié sur cluster
Reconstruction d'un environnement	une procédure écrite	jouée le 22 septembre depuis une destruction réelle (<code>RELEASE.md §9.5</code>)
Lecture des logs	<code>kubectl logs</code> , un pod à la fois, sans historique	data stream Elasticsearch, cinq tableaux de bord versionnés
Détection d'un incident	quelqu'un regarde un écran, ou personne	8 règles évaluées chaque minute
Mesure de la livraison	aucune	4 indicateurs DORA calculés sur 57 pipelines, tous renseignés

Trois de ces gains valent plus que les autres, parce qu'ils changent une propriété et pas seulement un chiffre.

« **Construire une fois, déployer partout** » est devenu vrai. Tant que l'URL de l'API était compilée dans le bundle, il fallait une image par environnement : l'image testée n'était pas celle déployée. Aujourd'hui Caddy fabrique un `/config.json` à partir de son environnement et le bundle le lit avant de démarrer. La même image, construite et scannée une fois, passe de staging à production sans être reconstruite.

La capacité de retour arrière est passée de nulle à réelle. Avant correction, `kubectl get deploy` affichait `READY 1/1 UP-TO-DATE 1 AVAILABLE 1` pendant que le Deployment pointait sur une image inexistante — une supervision branchée sur ce résumé n'aurait levé aucune alerte. C'est le genre de défaut qu'aucune relecture de YAML ne trouve, et que seule une campagne sur cluster révèle.

La reconstruction d'un environnement a été jouée, pas seulement écrite. Le 22 septembre, le namespace de staging a été détruit puis reconstruit depuis le seul dépôt : Ansible en `ok=23` `changed=0` , six ressources recréées par Terraform, rollout des deux Deployments en 11 secondes, API de nouveau servie. L'exercice a révélé trois écarts, écrits dans `RELEASE.md §9.5` : le namespace n'avait jamais été sous gouvernance Terraform, une étape de la procédure était incomplète, et l'état Terraform a dû être substitué puis migré. La restauration de *données*, elle, n'a pas d'objet tant que la base vit en mémoire.

7.2 Les gains de sécurité

Sujet	Avant	Après	État de la preuve
Scans Trivy	informatifs (<code>--exit-code 0</code>)	bloquants sur HIGH et CRITICAL	en CI depuis le 19 septembre
Dependency-Check	bloquant, vert, et n'analysant aucun jar	analyse 82 dépendances	vérifié sur le poste, non commité
Écart Dependency-Check / Trivy	inexpliqué	expliqué : le premier ne lisait rien	journal <code>--info</code> du plugin
Scan d'image	après le push : l'image refusée restait au registry	avant le push : une image refusée n'est jamais publiée	testé par les faux programmes, non commité
Rapports de scan	à relire dans le journal du job	JSON et tableau lisible, en artefacts de <code>trivy-fs</code> et <code>package-*</code>	produits sur le poste, non commité
Vue d'ensemble des vulnérabilités	aucune	collecteur <code>collect_security.py</code> , index <code>microcrm-security</code> , tableau de bord	alimenté à la main le 2 octobre
Exceptions	un fichier vide, parce que le scan était vide	7 exceptions Trivy et 3 Dependency-Check, toutes justifiées, échéance 31/12/2026	relues dans les deux fichiers
Supervision de la sécurité	aucune	2 règles d'alerte, déclenchées et rétablies	l'une sur staging, l'autre sur un conteneur local
CVE de Jackson et de Log4j	5 HIGH sur l'image publiée, 1 CVE Log4j de score ≥ 7	Jackson 2.21.7 et Log4j 2.25.5 forcés	0 constat sur le jar et l'image locale

Ce que ces gains apportent à la fiabilité d'ensemble : **une faille connue est désormais arrêtée au plus tôt, et de façon visible**. Deux des trois échecs de `develop` en sont la démonstration, même s'ils ne flattent pas les indicateurs : le 1^{er} octobre, la chaîne s'est arrêtée sur `trivy-fs`, puis sur le scan de l'image du back.

Recommandations pour renforcer le suivi et la correction des vulnérabilités, par ordre d'utilité :

1. **Brancher le collecteur de sécurité dans le pipeline**. Aujourd'hui, l'index `microcrm-security` n'est alimenté qu'à la main : l'historique du tableau de bord ne grandira pas tout seul. Un job qui exécute `collect_security.py` après les scans le rendrait vivant.

2. **Collecter le rapport Dependency-Check réel.** Le format produit par le plugin n'a pas été confronté au collecteur ; tant que ce n'est pas fait, les douze exceptions Spring n'apparaissent dans aucun tableau de bord.
3. **Suivre l'échéance des exceptions.** Elles expirent toutes le 31 décembre 2026. La tuile « jours avant la première échéance » existe ; une alerte à 30 jours éviterait de le découvrir par un pipeline rouge.
4. **Migrer vers Spring Boot 4** avant cette date : c'est la seule sortie qui retire les douze exceptions au lieu de les renouveler.
5. **Ajouter npm audit à l'étape security**, seul angle mort de dépendances du projet.
6. **Supprimer du registry l'image back:5bf1d6a2**, publiée avec ses cinq CVE avant le correctif d'ordre du scan.
7. **Scanner périodiquement les images déjà déployées.** Une image acceptée le 23 septembre peut être déclarée vulnérable le lendemain : un scan planifié des images en service, indépendant des commits, comblerait cet écart.
8. **Activer la sécurité d'Elasticsearch**, sans quoi l'outil qui supervise la sécurité est lui-même ouvert.

7.3 Observations sur la phase de release

Une release, ici, consiste à poser un tag de version sur un commit de `main`. Le pipeline de tag ne reconstruit rien : il **promeut** les images déjà construites et scannées pour ce commit, en leur ajoutant le numéro de version.

Le tag v1.0.0 n'a jamais abouti, et la cause est établie. Posé le 22 septembre sur le commit `5d459fb`, son pipeline a échoué sur `test-front`, avant la promotion : `Can not find the binary /opt/google/chrome/chrome`. Le runner du projet tourne sur Apple Silicon. L'image de test, alors désignée par son tag, y résolvait vers sa variante arm64, qui n'embarque pas Chrome. Le correctif — désigner l'image par son empreinte amd64 — est arrivé une heure et demie plus tard, dans un commit postérieur au tag. Aucune image `1.0.0` n'existe au registry, aucune Release n'a été créée. Un tag ne se déplace pas : `v1.0.0` reste où il est, comme trace.

La release 1.0.1 est préparée, pas publiée. Les fichiers de version portent `1.0.1` (`back/build.gradle`, `front/package.json`, `front/package-lock.json`, `appVersion` du chart Helm), et `check_version.sh --version v1.0.1` sort en `0`. Rien n'est commité, aucun tag n'est posé, aucune image `1.0.1` n'existe.

Observation pendant la phase de release	Amélioration apportée	État
Le pipeline de tag a échoué sur une image d'outillage résolue vers la mauvaise architecture	image désignée par son empreinte amd64	en CI, <code>test-front</code> repasse

Observation pendant la phase de release	Amélioration apportée	État
Un tag disait « cette version existe » sans dire quelles images la portent	job <code>release</code> : crée la Release GitLab après les deux promotions, avec le commit, le pipeline et les tags d'image	écrit, jamais exécuté
Une image refusée par le scan pouvait être promue, puisqu'elle était déjà au registry	scan avant le push	écrit et testé, non commité
<code>package-lock.json</code> portait <code>0.0.0</code> à deux endroits	aligné, et contrôlé par <code>version-consistency</code>	vérifié sur le poste
La procédure de release ne disait pas quoi faire quand la promotion échoue	<code>RELEASE.md</code> §7 réécrit : prérequis, séquence, quatre causes d'échec	relu
Un pipeline de <code>main</code> sain s'affiche « Blocked », ce qui se lit comme un échec	expliqué dans la procédure et dans ce rapport (§2.3)	documenté

Ce que ces observations enseignent. La promotion est le bon mécanisme : l'image mise en production est exactement celle qui a été testée et scannée. Mais elle rend la release dépendante de tout ce qui précède. Une image d'outillage mal résolue, une CVE publiée la veille, un numéro de version oublié dans un fichier : chacun de ces défauts, sans rapport avec l'application, arrête la release. D'où trois recommandations qui répondent directement à ce qui a été observé :

1. **Jouer la release 1.0.1 de bout en bout.** Le job `release` ne se prouve qu'en poussant un tag. La séquence est écrite dans `RELEASE.md` §7.
2. **Tester le chemin de tag sans attendre une vraie version**, par exemple avec un tag de pré-version, pour que l'échec de `v1.0.0` ne se reproduise pas le jour d'une livraison attendue.
3. **Rendre le déploiement de staging automatique**, en gardant celui de production manuel : le délai de mise en production cessera de mesurer un clic.

7.4 Recommandations d'amélioration continue

Classées par ce qu'elles débloquent, et non par leur difficulté. L'effort est noté S (quelques heures), M (quelques jours).

Ce qui est déjà fait

Piste annoncée dans les versions précédentes de ce rapport	Date	État
Un runner disponible	22/09	fait — runner auto-hébergé, plus aucune minute facturée

Piste annoncée dans les versions précédentes de ce rapport	Date	État
Un premier <code>deploy-staging</code> mené à son terme	22/09	fait — les quatre indicateurs sont renseignés
Retirer <code>allow_failure</code> contrôle par contrôle	14/09 et 19/09	fait — seul <code>k6-load</code> reste non bloquant
Jouer la reconstruction complète d'un environnement	22/09	fait — <code>RELEASE.md §9.5</code>
Activer <code>--scan</code> dans <code>package-back</code> et <code>package-front</code>	02/10	écrit et testé sur le poste, non commité
Un job exécutant <code>collect_dora.py</code>	02/10	écrit (<code>dora-metrics</code>), jamais exécuté dans un pipeline
Alerting sur les erreurs applicatives	02/10	fait sur le cluster — 8 règles, fichiers non commités

Ce qui remet la chaîne en marche

Piste	Effort	Ce que ça change pour Orion
Commiter et pousser le travail du 2 octobre	S	Seul moyen de savoir si les jobs nouveaux fonctionnent dans un runner.
Remettre <code>develop</code> au vert, puis publier la 1.0.1	S	Les correctifs de sécurité et les traces arrivent enfin en service.
Déployer l'image instrumentée en staging	S	La latence de l'API et trois règles d'alerte cessent d'être une démonstration.
Recaler le seuil de <code>perf-api-p95</code> sur un trafic réel	S	Le seuil actuel repose sur 401 transactions d'un conteneur local.

Ce qui rend la mesure fiable

Piste	Effort	Ce que ça change pour Orion
<code>metrics-server</code> , puis des métriques de ressources	M	Donne CPU et mémoire — de quoi vérifier les <code>resources</code> , aujourd'hui estimées, et le surcoût de l'agent.
Étendre la collecte des logs à la production	S	La production cesse d'être invisible.
Une sonde externe de disponibilité	S	Remplace la « présence de logs » par un vrai taux de disponibilité.

Piste	Effort	Ce que ça change pour Orion
Un pont entre <code>microcrm-alerts</code> et <code>notify.py</code>	S	Une alerte prévient quelqu'un. C'est un petit programme, sans licence payante.
Alimenter les index DORA et sécurité depuis la CI	M	Les tableaux de bord cessent de dépendre d'un geste manuel. Suppose un Elasticsearch joignable par le runner.
Un runner dédié pour <code>k6-load</code>	M	Rend le seuil de charge défendable, donc bloquant.
<code>kubeconform</code> dans <code>lint-k8s</code>	S	Valide les manifestes contre le schéma de l'API, hors ligne.

Ce qui lève les limites structurelles

Piste	Effort	Ce que ça change pour Orion
Remplacer HSQLDB par PostgreSQL persistant	M	Lève deux limites d'un coup : le plafond à 1 replica et les 16 s de coupure sur perte de pod. Impose Flyway ou Liquibase.
Migrer vers Spring Boot 4	M	Retire les douze exceptions de sécurité avant leur échéance.
ILM sur les data streams et les index	S	Borne la croissance des logs, des alertes et des constats.
Activer la sécurité d'Elasticsearch	M	Condition pour que la stack sorte d'un poste de développement.
Un CNF qui implémente les NetworkPolicy	S	Le cloisonnement réseau cesse d'être décrit pour devenir effectif.
<code>jlink</code> sur l'image du back	M	390 Mo → ~150 Mo, en ne gardant que les modules réellement utilisés.

Le contexte d'Orion derrière cet ordre. Une petite équipe ne peut pas tout mener de front. La première série ne demande aucun investissement : elle met en service ce qui est déjà écrit. La deuxième transforme une supervision de démonstration en outil d'exploitation. La troisième engage des choix d'architecture, et c'est elle qui appelle une décision du CTO.

8. Ce qui manque, et ce qui n'est pas prouvé

Cette liste est donnée sans enrobage : elle est ce qu'un jury est en droit d'attaquer, et il vaut mieux qu'elle vienne du rapport que de la lecture.

8.1 Ce qui a été fait le 2 octobre et n'est pas encore prouvé en conditions réelles

- **Rien n'est commité ni poussé.** Les jobs `release` et `dora-metrics`, le scan avant le push, les rapports Trivy en artefacts et le correctif de Dependency-Check n'ont tourné dans aucun pipeline. Ils sont vérifiés par les tests de scripts et par des exécutions sur le poste.
- **Le pipeline de `develop` est rouge sur ses trois dernières exécutions :** `#2892321711` (29 septembre), `#2901472002` et `#2902337581` (1^{er} octobre). Relevé le 2 octobre par l'API GitLab. Causes : aucun runner disponible pour le premier, `trivy-fs` et `terraform-plan` pour le deuxième, `package-back` pour le troisième. La cause de l'échec de `terraform-plan` n'a pas été recherchée.
- **La release 1.0.1 est préparée, pas publiée.** Le job `release` n'a jamais créé de Release ; cela ne se prouve qu'en poussant un tag.
- **Le fichier modifié du pipeline n'a pas été validé par GitLab.** Son lint n'accepte un contenu local qu'avec un jeton. À la place : chargement de chaque fichier, résolution des héritages et contrôle des dépendances entre jobs sur six scénarios, sans erreur.
- **Les scans d'image ont été éprouvés contre Docker Desktop**, pas contre le démon Docker d'un runner. Le push réel vers le registry n'a pas été rejoué.
- **Les trois règles d'alerte fondées sur les traces sont prouvées sur un conteneur local**, pas sur les pods déployés.
- **L'historique du tableau de bord de sécurité est rejoué**, et aucun rapport Dependency-Check réel n'y a été collecté.
- **La tenue des alertes dans la durée n'est pas connue** : aucun recul sur les faux positifs. L'une d'elles sonne au réveil d'un poste mis en veille.

8.2 Ce qui manque

- **Les traces ne sont pas déployées.** Le code est là, APM Server tourne, et aucun pod du cluster n'émet de trace : les images en service datent du 23 septembre.
- **Aucune métrique de ressources.** Ni CPU, ni mémoire, ni disque.
- **La disponibilité n'est pas mesurée.** Elle est déduite de la présence de logs, en staging seulement. La production n'est pas collectée.
- **Aucune notification ne sort de Kibana**, et l'alerting ne se surveille pas lui-même.
- **Aucune rétention des logs.** Pas d'ILM ; les seuils disque d'Elasticsearch (85 / 90 / 95 %) mettront l'index en lecture seule bien avant que le volume de 5 Gio ne soit plein.
- **Sécurité d'Elasticsearch désactivée**, et Kibana accessible sans authentification derrière un simple `port-forward`.
- **Douze CVE de Spring Framework sont acceptées par exception** jusqu'au 31 décembre 2026 (§4.4). L'analyse est écrite ; elle n'est pas une preuve d'innocuité.
- **Aucun `npm audit`**, et la porte de qualité Sonar ne tourne que sur `main`.
- **Le comportement sous incident n'a été exercé qu'à la main.** Les échecs mesurés en CI portent sur l'accès au cluster, pas sur l'application : le rollback automatique de `deploy.sh` n'a donc été déclenché pour de vrai que lors de la campagne locale de `K8S.md` §14. Le rollback

joué en production le 23 septembre appelait `rollback.sh` depuis `rollback-production`, un job manuel — ce n'est pas le même chemin de code.

- **La reconstruction d'environnement a été jouée avec trois écarts.** Images chargées localement et non tirées du registry, état Terraform substitué puis migré depuis un poste, et non depuis un job (`RELEASE.md §9.5`).
- **Les NetworkPolicy sont décrites, pas prouvées.** Le CNI par défaut de minikube ne les implémente pas et les ignore sans rien signaler. Celle d'APM Server n'a d'ailleurs pas été appliquée.
- **Le chart Helm n'a jamais déployé.** Il est rendu, linté, comparé au rendu Kustomize et accepté en `--dry-run=server`. Rien de plus.
- **La production n'existe pas.** L'environnement `production` vise le même minikube dans un autre namespace ; il démontre qu'un second environnement se décrit par les mêmes modules et d'autres valeurs, pas qu'une production existe.

9. Où retrouver les preuves

Ce rapport ne remplace pas les documents techniques du dépôt : il les résume. Chaque chiffre cité y est accompagné de la commande qui l'a produit et de la sortie observée.

Document	Ce qu'il porte
<code>MONITORING.md</code>	La stack ELK, les indicateurs DORA (§9), les traces et leur relevé (§10)
<code>k8s/elk/alerting/README.md</code>	Les huit règles, la justification de chaque seuil, les preuves de déclenchement
<code>k8s/elk/alerting/rules/</code>	Les règles elles-mêmes, une par fichier, requête comprise
<code>k8s/elk/dashboards/README.md</code>	Chaque panneau, confronté à l'agrégation Elasticsearch équivalente
<code>back/config/dependency-check/suppressions.xml</code>	Les douze exceptions Spring et leur analyse, condition par condition
<code>.trivyignore.yaml</code>	Les sept exceptions Trivy, justifiées et datées
<code>RELEASE.md §7 et §9.5</code>	La procédure de release, le précédent <code>v1.0.0</code> , la reconstruction du 22 septembre
<code>K8S.md §14</code>	Les deux campagnes de déploiement réel, commandes et sorties
<code>QUALITY.md</code>	Les six outils, leurs seuils, et ce qu'ils ont trouvé
<code>SCRIPTS.md</code>	Les scripts, leurs codes de sortie, et comment ils sont testés

Document	Ce qu'il porte
TERRAFORM.md , ANSIBLE.md , HELM.md	L'infrastructure, ses frontières et ses limites assumées
ARCHITECTURE.md §9 à §11	Pourquoi l'option locale, sa transposition au cloud, et ses angles morts
documentation-infrastructure.md	Le document jumeau : architecture et procédures
docs/captures/	Les captures d'écran : pipelines, environnements, tableaux de bord, alertes

Sur **docs/captures/** . Vingt-deux fichiers s'y trouvent, dont dix sont repris dans ce rapport.

Reprises dans ce rapport	Section
kibana-dora-quatre-indicateurs-2026-10-02.png	§2.3
kibana-dora-non-mesurable-affiche-na-aout-2026-10-02.png	§2.3
pipelines-liste-apres-deploiements-2026-09-23.png	§2.3
kibana-apm-services-2026-10-02.png	§5.3
kibana-apm-transactions-microcrm-2026-10-02.png	§5.3
kibana-securite-vulnerabilites-exceptions-erreurs-2026-10-02.png	§6.4
kibana-disponibilite-sondes-taux-demarrages-2026-10-02.png	§6.4
kibana-supervision-volume-latence-erreurs-2026-10-02.png	§6.4
kibana-alertes-liste-des-regles-2026-10-02.png	§6.5
kibana-alertes-suivi-declenchements-2026-10-02.png	§6.5

Les douze autres ne sont pas reprises, et il faut dire pourquoi. Deux sont les anciennes captures Kibana du 19 septembre (...-90j.jpg), remplacées par celles du 2 octobre : leur capture DORA montre l'état d'avant le déblocage. Quatre montrent les environnements GitLab et le commit du 23 septembre. Six montrent le détail des graphes de jobs de deux pipelines antérieurs. Aucun passage du texte n'en avait besoin, et une galerie d'images sans propos ne prouve rien. Elles restent dans le dépôt pour qui voudrait vérifier un pipeline étape par étape.

10. Glossaire

Terme	Signification
APM	<i>Application Performance Monitoring</i> : mesure du temps que met l'application à répondre, requête par requête.

Terme	Signification
Artefact	Fichier produit par un job du pipeline et conservé : un rapport, une image, un jar.
CI/CD	Intégration et déploiement continus : les contrôles et la livraison automatisés à chaque modification.
CVE	Identifiant public d'une faille de sécurité connue. Sa gravité va de LOW à CRITICAL.
CVSS	Note de gravité d'une CVE, de 0 à 10. Le pipeline s'arrête à partir de 7.
DORA	<i>DevOps Research and Assessment</i> : les quatre indicateurs de performance de livraison (§2.1).
ELK	Elasticsearch (stockage), Logstash ou Filebeat (collecte), Kibana (affichage) : la stack de supervision.
Image	Paquet contenant l'application et tout ce qu'il lui faut pour tourner.
Job, pipeline	Un job est un contrôle ou une action automatique ; un pipeline est la suite de jobs lancée par un commit.
KPI	Indicateur clé de performance.
MTTR	Temps moyen de rétablissement après un échec.
Namespace	Cloison logique d'un cluster Kubernetes : ici, un par environnement.
p50, p95, p99	Percentiles. « p95 = 5 ms » veut dire que 95 % des requêtes répondent en moins de 5 ms.
Pod	Unité d'exécution de Kubernetes : une instance en marche de l'application.
Registry	Entrepôt où sont rangées les images.
Release, tag	Un tag est une étiquette de version posée sur un commit ; la release est la version publiée qui en découle.
Rollback	Retour à la version précédente.
Runner	Machine qui exécute les jobs du pipeline.
Staging	Environnement de recette, où une version est essayée avant la production.
Trace	Enregistrement du parcours d'une requête dans l'application, avec la durée de chaque étape.